Lecture 13. Introduction to Cell-Probe Model

Spring 2026, *Theory of Computation*, IIIS, Tsinghua

"Were-you-last?" Game

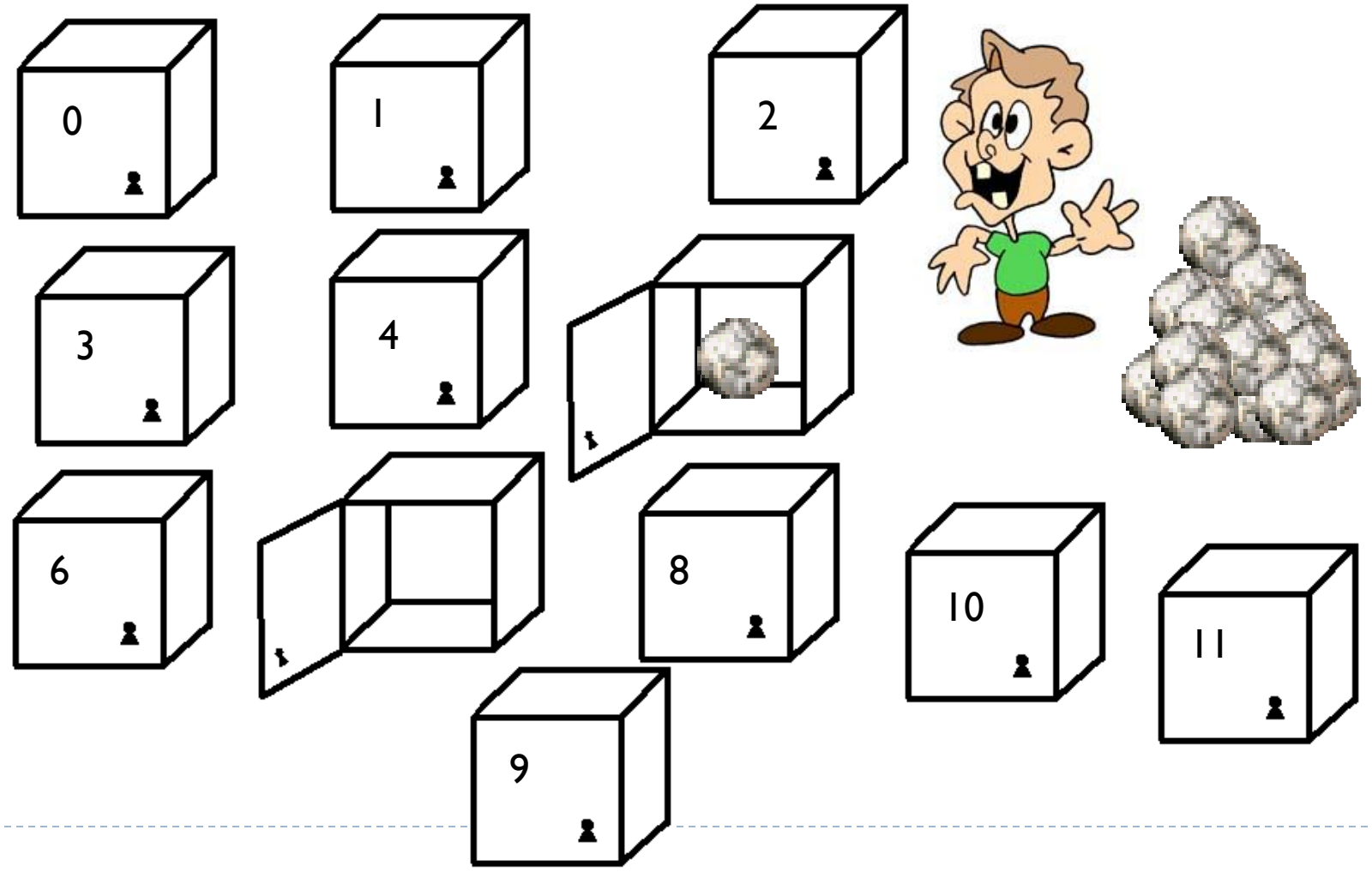
– a very simple example

- *m* players of Dream Team wait in separate cubicles
- One by one, they are taken to a *game room*
  - They don't know the order
- When a player leaves the game room, he is asked if he was the last of the *m* players to go there and must answer correctly



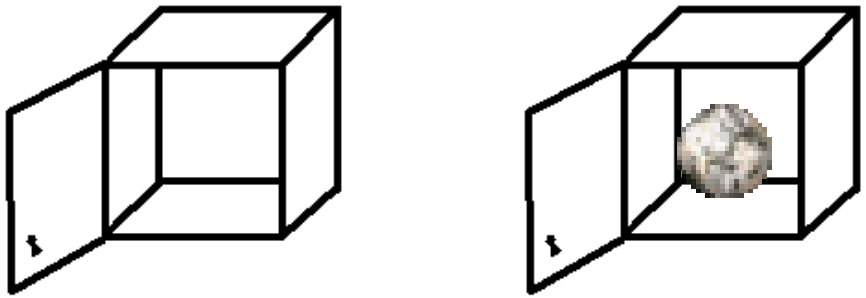


Inside the Game Room



Inside the Game Room

- There are many boxes
- Every player can open at most *t* boxes
  - He can put a pebble in an empty box or remove the pebble from a full box
  - He must close a box before opening the next one





Winning the Game (*m players, t boxes for each player*)

- The Dream Team win if all players answer correctly
- Assume there is their adversary, *Hannibal*, who knows their strategies, arranging the order of their entrance to the game room
- Hannibal wins if some player answers wrong, and the Dream Team will faces a gory destiny
- Is the Dream Team doomed?



“Were-you-last?” Game

- m players
- total k bits (*no bound*)
- each player can only read/write ≤t bits



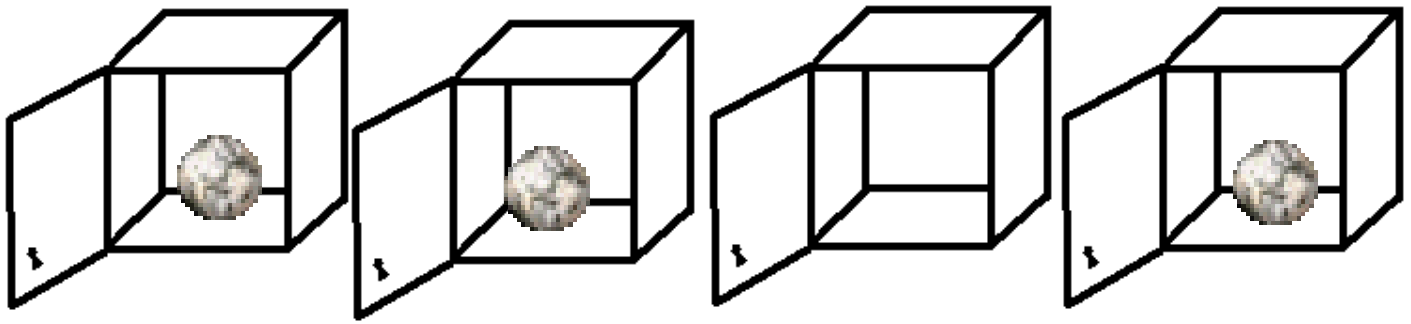
- so it depends on how m and t relate
  - it’s a dynamic data structure problem



A counter: $t=\Theta(\log_2 m)$

- The bits represent a binary number from 0 to 2^t-1
- Strategy:

```
if (Counter==0), set Counter←m-1;
        else Counter--;
if (Counter==0) return true; else return false;
```



$$=1101=13$$

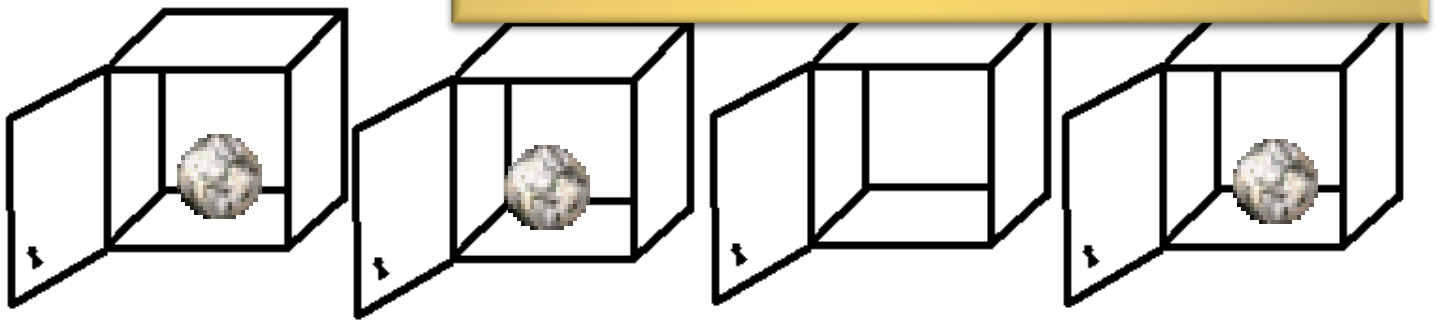


A counter: $t=\Theta(\log_2 m)$

- The bits represent a binary number from 0 to 2^t-1
- Strategy:

```
if (Counter==0), set Counter←m-1;
        else Counter--;
if (Counter==0) return true; else return false;
```

Can we make t smaller?



$=1101=13$



Make $t=O(\text{loglog } m)$

- Any ideas?

The decremental counter affects many bits only when xxx100000 → xxx011111



Make $t=O(\log\log m)$

- Look at the "blocks" of 0s and 1s in binary numbers
- e.g. 110111000001 1000 1 has 7 blocks of 0s and 1s



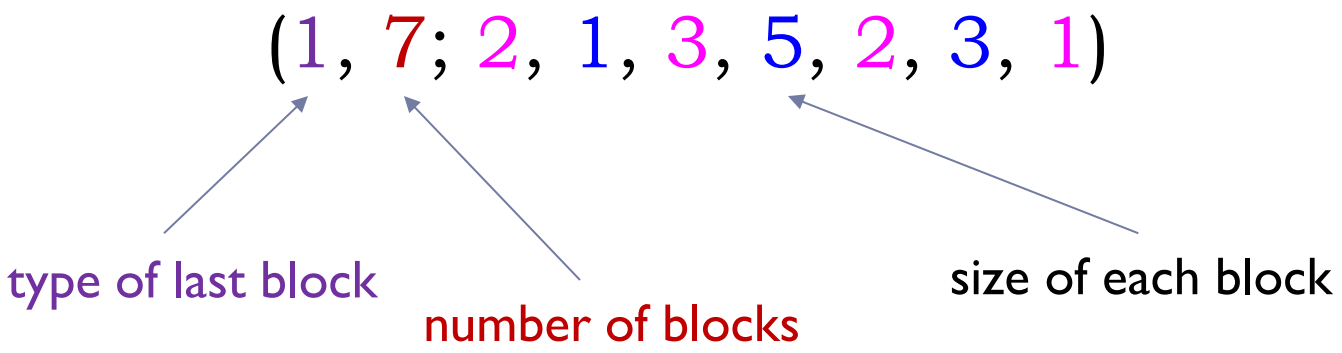
Make $t=O(\log\log m)$

- Look at the "blocks" of 0s and 1s in binary numbers
- e.g. 1101110000011000 has 7 blocks of 0s and 1s
- So we need to keep track of:
  - The number of blocks
  - The length of each block
  - First block is 0 or 1? (from left or from right)



Make $t=O(\log\log m)$

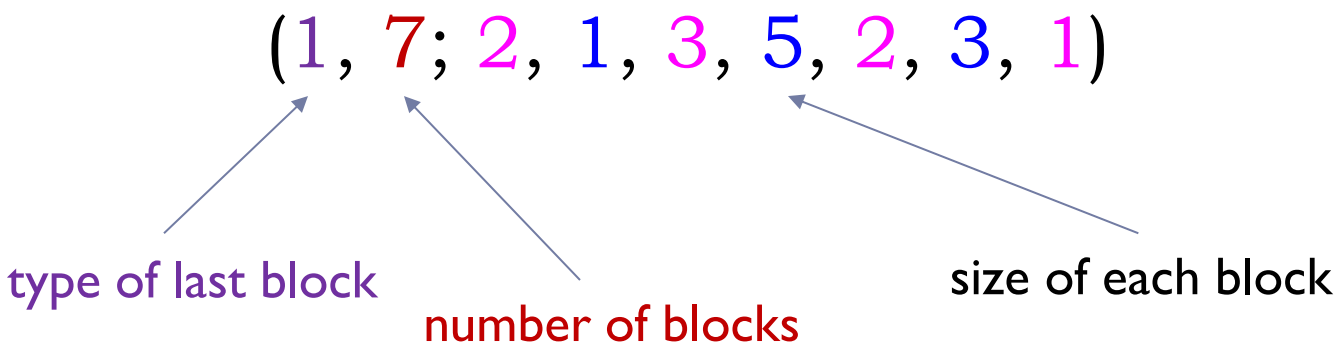
- e.g. 110111000001111000 has 7 blocks of 0s and 1s





Make $t=O(\text{loglog } m)$

- e.g. 1101111000011001 has 7 blocks of 0s and 1s



- Each entry can be specified using $\leq \lceil \text{loglog } m \rceil$ bits
- How many bits we need to read/write when decreasing the number by 1?



1101111000001111000111

we only need to change blocks in the end

m	m-1	last bits	#blocks	#changed block
…0001	…0000	change	-1	2
…1111	…1110	change	+1	2
… 01000	…00111	change	-1	3
… 11000	…10111	change	+1	3

length of each block: ≤log m, requiring loglog m bits to represent
So $t \le 8\lceil \log\log m \rceil + O(1)$, since we need to read/write ≤ 3 blocks and change #blocks and the one bit type of last block



Last problem

- How can the first player set the counter to m?
  - if $m=2^i$ or 2^i+1, then it's easy
  - How about arbitrary m?

m=11010101100110001



Last problem

- In *["Cell probe complexity-a survey", Peter Miltersen]:*
- You have a strategy which answers correctly when the counter begins at some status m'
  - block representation of m
- If the boxes C begins at status 0, you can assume it begins at m', that is, you assume you are always working on C'=C⊕m'

- Other possible way:
  - using first log m players to set the counter to m-log m



- What about t=o(loglog m), can the Dream Team still win?



- What about t=o(loglog m), can the Dream Team still win?
- If we can reading is free and writing is expensive, we can use Gray code:
  - 1-bit write



	Gray code			
	4	3	2	1
0	0	0	0	0
1	0	0	0	1
2	0	0	1	1
3	0	0	1	0
4	0	1	1	0
5	0	1	1	1
6	0	1	0	1
7	0	1	0	0
8	1	1	0	0
9	1	1	0	1
10	1	1	1	1
11	1	1	1	0
12	1	0	1	0
13	1	0	1	1
14	1	0	0	1
15	1	0	0	0



- What about t=o(loglog m), total read/write bits?

- NO! If $t \leq 0.4$ loglog m, the players lose



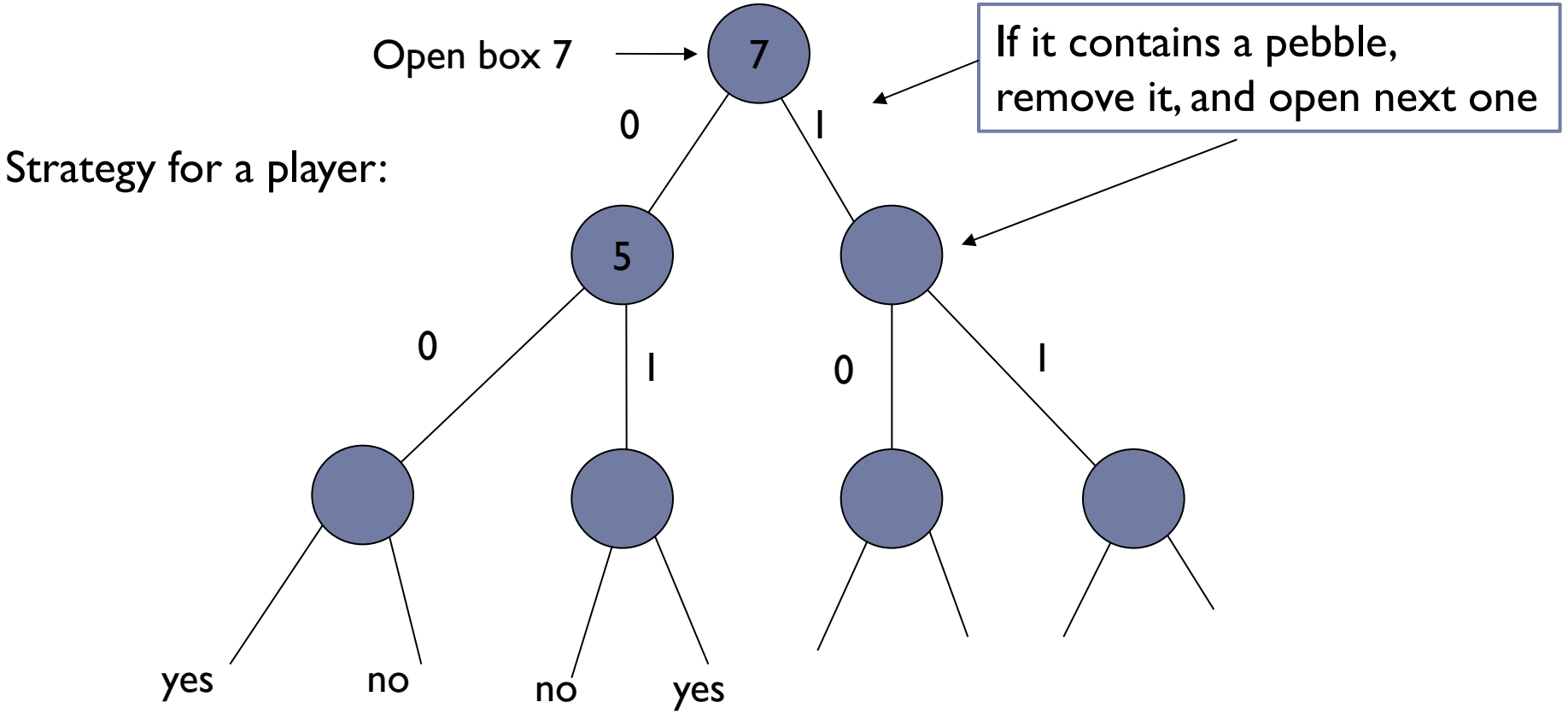
When $t \leq 0.4 \log \log m$,

- Define S_i to be the set of all bits that player i may read/write
- Then $|S_i| \leq l = 2^t - 1$ (WHY?)



When $t \le 0.4 \log \log m$,

- Define S_i to be the set of all bits that player i may read/write
- Then $|S_i| \le l = 2^t - 1$ (WHY?)





“Symmetric” strategy

- If the strategy for every player is the same, then $S_i=S$ for all player i
  - S_i to be the set of all bits that player i may read/write
  - $|S_i| \le l = 2^t - 1$

- why cannot t<loglog m?



“Symmetric” strategy

- If the strategy for every player is the same, then $S_i=S$ for all player i
  - S_i to be the set of all bits that player i may read/write
  - $|S_i|\leq l=2^t-1$
- why cannot t<loglog m?
  - then $|S|<2^t<\log m$
  - possible states for all bits in |S|: $2^{|S|}<m$
  - when all players $1,\ldots,m$ come in, there are i and j after which all bits are in the same state. So they cannot distinguish 1,…,i,..,j,...,m and 1,...,i,j+1,...m

- what about asymmetric strategies?
  - different strategy for each player



Sunflower lemma

- A sunflower:
  - a collection of sets $S_1,\ldots,S_p$ so that $S_i \cap S_j$ is the same for each pair of different sets S_i and S_j

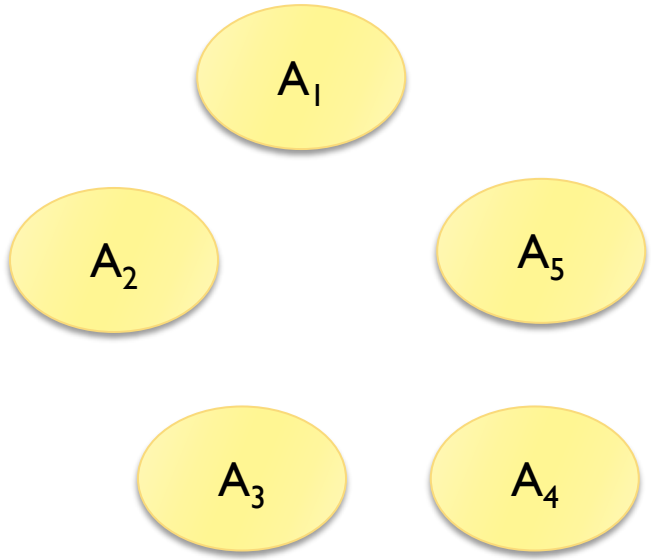
- Let $S_1,\ldots,S_m$ be a collection of sets each of cardinality $\leq l$. If $m>(p-1)^{l+1}\, l!$, then the collection contains a sunflower with p sets





- A sunflower:
  - a collection of sets $S_1,\ldots,S_p$ so that $S_i \cap S_j$ is the same for each pair of different sets S_i and S_j
- Let $\mathcal{F}=\{S_1,\ldots,S_m\}$ be a collection of sets each of cardinality $\leq \boldsymbol{l}$. If $m>(p-1)^{l+1}\, \boldsymbol{l}!$, then the collection contains a sunflower with p sets

Induction on $\boldsymbol{l}$: for $\boldsymbol{l}=1$, p sets of different single elements form a flower
For $\boldsymbol{l}\geq 2$,
- take a maximal collection $A_1,\ldots,A_t$ of pairwise disjoint members in $\mathcal{F}$
- if $t\geq p$, they form a flower with empty core



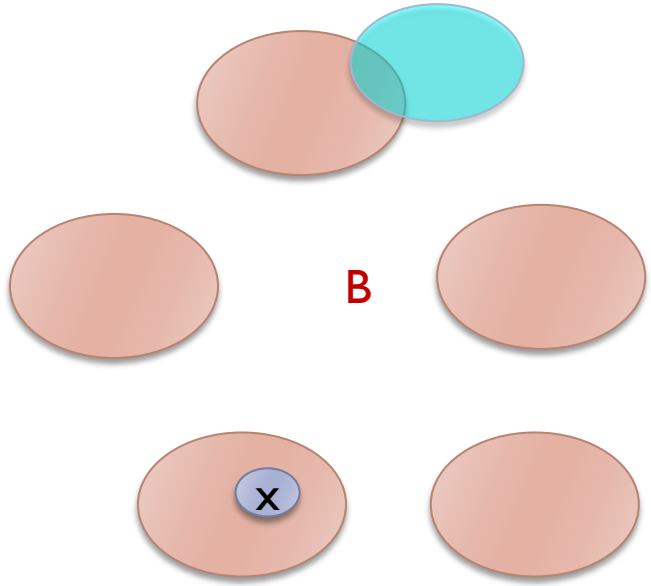


- A sunflower:
  - a collection of sets $S_1,\ldots,S_p$ so that $S_i \cap S_j$ is the same for each pair of different sets S_i and S_j
- Let $\mathcal{F}=\{S_1,\ldots,S_m\}$ be a collection of sets each of cardinality $\leq l$. If $m>(p-1)^{l+1}\, l!$, then the collection contains a sunflower with p sets

Induction on l: for $l=1$, p sets of different single elements form a flower
For $l\geq 2$,

- take a maximal collection $A_1,\ldots,A_t$ of pairwise disjoint members in $\mathcal{F}$
- if $t\geq p$, they form a flower with empty core
- if $t<p$, let $B= A_1 \cup \ldots \cup A_t$, then $|B|\leq (p-1)l$, and B intersects every set in $\mathcal{F}$
- some element x in B must be contained in

$$> \frac{(p-1)^{l+1} l!}{(p-1) l} = (p-1)^l (l-1)! \text{ sets in } \mathcal{F}$$





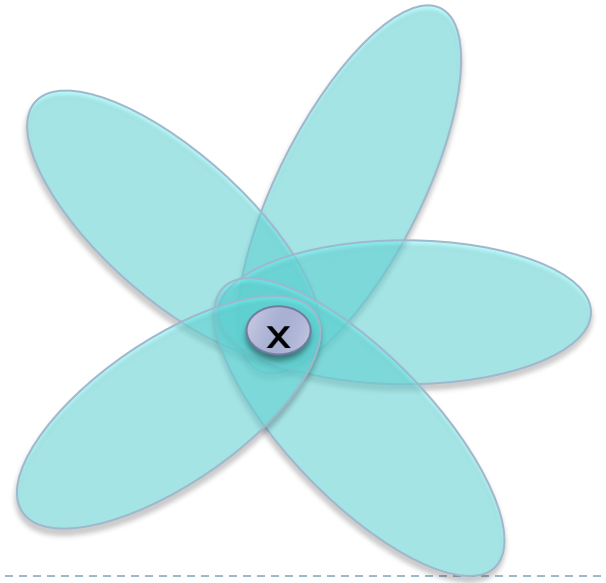
- A sunflower:
  - a collection of sets $S_1,\ldots,S_p$ so that $S_i \cap S_j$ is the same for each pair of different sets S_i and S_j
- Let $\mathcal{F}=\{S_1,\ldots,S_m\}$ be a collection of sets each of cardinality $\leq \boldsymbol{l}$. If $m>(p-1)^{l+1}\,\boldsymbol{l}!$, then the collection contains a sunflower with p sets

Induction on $\boldsymbol{l}$: for $\boldsymbol{l}=1$, p sets of different single elements form a flower
For $\boldsymbol{l}\geq 2$,

- take a maximal collection $A_1,\ldots,A_t$ of pairwise disjoint members in $\mathcal{F}$
- if $t\geq p$, they form a flower with empty core
- if $t<p$, let $B= A_1 \cup \ldots \cup A_t$, then $|B|\leq (p-1)\boldsymbol{l}$, and B intersects every set in $\mathcal{F}$
- some element x in B must be contained in

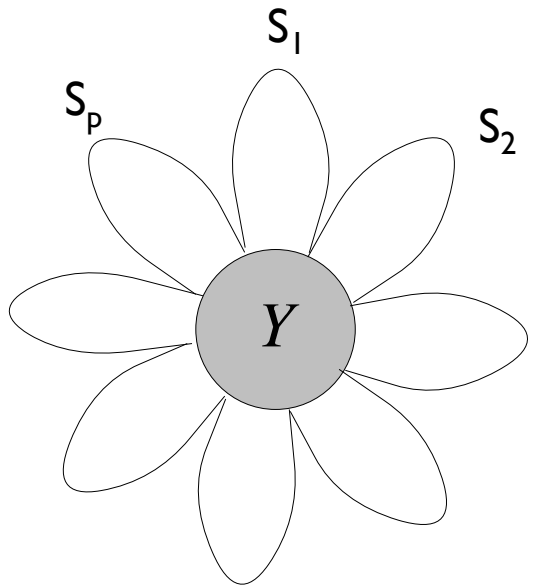
$$> \frac{(p-1)^{l+1}l!}{(p-1)l} = (p-1)^l(l-1)! \text{ sets in } \mathcal{F}$$

- delete x from them, by induction hypothesis, these sets form a sunflower with p sets
- add x to them to get a sunflower in $\mathcal{F}$





- Define S_i to be the set of all bits that player i may read/write
  - Then $|S_i| \leq \ell = 2^t - 1$
- For $S_1, \ldots, S_m$ $(m > (p-1)^{\ell+1}\, \ell!)$, we find a sunflower with p sets,
  - assume they are $S_1, \ldots, S_p$





- Define S_i to be the set of all bits that player i may read/write
  - Then $|S_i| \le l = 2^t - 1$
- For $S_1, \ldots, S_m$, we find a sunflower with p sets,
  - assume they are $S_1, \ldots, S_p$

- Hannibal first arrange other m-p players, then these p players
- only the center of the sunflower is meaningful for these players



- Define S_i to be the set of all bits that player i may read/write
  - Then $|S_i| \le l = 2^t - 1$
- For $S_1, \ldots, S_m$, we find a sunflower with p sets,
  - assume they are $S_1, \ldots, S_p$

- Hannibal first arrange other m-p players, then these p players
- only the center of the sunflower is meaningful for these players
  - possible #status for the center: $\le 2^l$
  - So if $p > 2^l$, when p players comes in, two players i and j will leave the same status
  - so they cannot distinguish:

    1, ..., i, i+1, ..., j, j+1, ..., p

    1, ..., i, j+1,..., p



Let $S_1,\ldots,S_m$ be a collection of sets each of cardinality $\leq l$. If $m>(p-1)^{l+1}\, l!$, then the collection contains a sunflower with p sets

- $|S_i|\leq l=2^t-1$
- When t= 0.4 loglog m,

$$l < (\log m)^{0.4}$$

$$\left(2^l\right)^{l+1} l^l = 2^{l^2+l+l\cdot\log l} < m$$

- So we can find a sunflower with $p>2^l$ sets



Summary

- A dynamic structure for a decremental counter
- simple counter:
  - space $O(\log m)$,
  - update time $O(\log m)$
- block counter:
  - space $O(\log m \cdot \operatorname{loglog} m)$,
  - update time $\Theta(\operatorname{loglog} m)$



Bounds for complexity

- Upper bound:
  - If we have found an algorithm with time complexity O(f(n)) for a language L, we can see the time complexity for L is O(f(n))
  - But there may be faster algorithm we have not known, so O(f(n)) is just an upper bound
- Lower bound:
  - Prove that any algorithm for L needs Ω(f(n)) time (or space)
  - Time (space) lower bound



Comparison-based Sorting

- **Comparison sort**
  - Only comparison of pairs of elements may be used to gain order information about a sequence
  - Hence, a lower bound on the number of comparisons will be a lower bound on the complexity of any comparison-based sorting algorithm

- The best worst-case complexity so far is $\Theta(n \lg n)$ (merge sort and heapsort)
- We prove a lower bound of $\Omega(n \lg n)$ for any comparison sort: thus merge sort and heapsort are optimal



Comparison-based algorithms

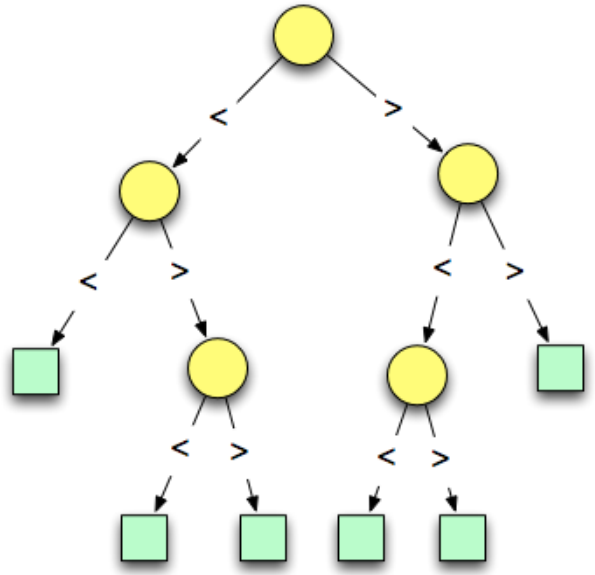
- For real weights
- We can only compare two numbers in each step
  - No bucket sort!

- Assume all weights are different!
  - compare a and b: a>b or a<b



Comparison-based algorithms

- For real weights
- We can only compare two numbers in each step
  - No bucket sort!
- Assume all weights are different!
  - compare a and b: a>b or a<b
- Can be seen as a decision tree:


< >
< >
< >

Comparison-based sorting

- Sort n distinct real numbers
- The decision tree:
  - a binary tree
  - n! leaf nodes, since there are n! possible permutations
  - so the depth of the tree is ≥ $\log_2(n!)$



Comparison-based sorting

- Sort n distinct real numbers
- The decision tree:
  - a binary tree
  - n! leaf nodes, since there are n! possible permutations
  - so the depth of the tree is ≥ $\log_2(n!)$
- By Stirling's approximation,

$$n! \sim \sqrt{2\pi n}\left(\frac{n}{e}\right)^n$$

so $\log_2(n!)=\Theta(n \cdot \log n)$



- Thus, the time lower bound for comparison-based sorting algorithm is $\Theta(n \log n)$

- How can we do better?



Beating the lower bound

- We can beat the lower bound if we don't base our sort on comparisons:
  - Counting sort for keys in [0..k], k=O(n)
  - Radix sort for keys with a fixed number of "digits"
  - Bucket sort for random keys (uniformly distributed)





- Lower bounds for decision problems are very hard in general
- Usually we can only prove it under particular model
- But there are many lower bounds (time/space) for data structure problems!



Cell probe model

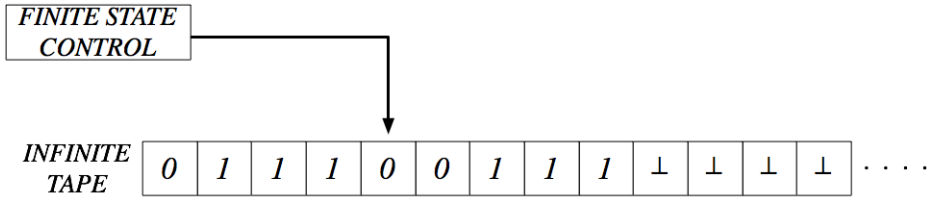
- Like random access machine (RAM) model
- We only count the number of memory accesses, not computations!

- For data structure problems, how many bits/cells we need to read/write?

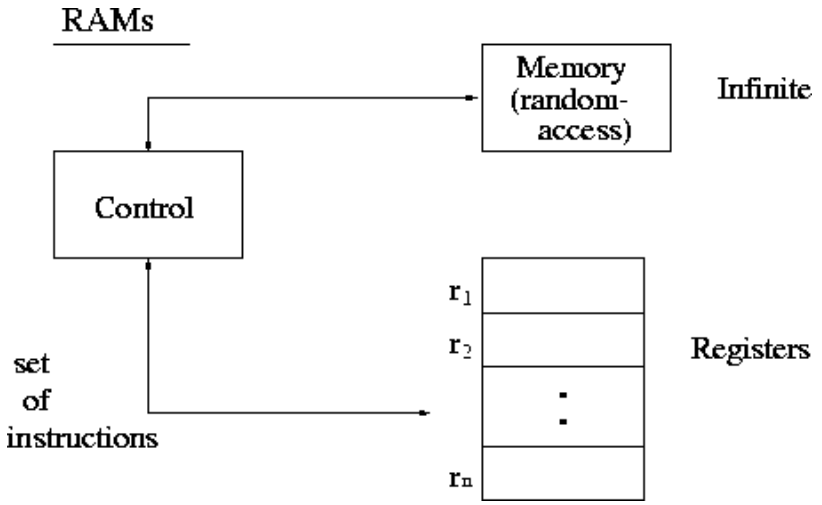


Random Access Machine

- In Turing machine, we need to move the head on the tape to the cell we need to read/write



- In a RAM, we can use address to find that cell
  - More like modern day computers





Cell probe model

- Computation cost: # accesses of memory cells
- Data structures:
  - Preprocessing phase: Given the input data, we can write in memory cells
  - Query phase: Determine whether a query datum is included in the input data. (Using the data from the preprocessing phase.)



Cell probe model

- Computation cost: # accesses of memory cells
- Dynamic data structures:
  - Preprocessing phase: Given the input data, we can write in memory cells
  - Update: The "input data" can be changed by a small amount each time
  - Query phase: Determine whether a query datum is included in the input data



Cell probe model

- History: proposed by Andrew Yao's 1981 paper "Should Tables Be Sorted?"

- Information Retrieval Problem:
  - Input: a set S of integers
  - Query: an integer x, if $x \in S$?

- Consider the number of probes for each query



Cell probe model

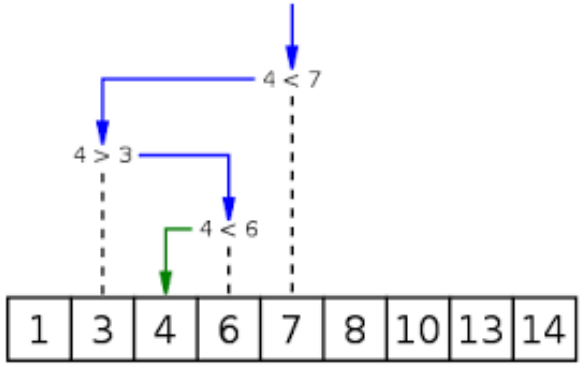
- Information Retrieval Problem:
  - Input: a set S of integers
  - Query: an integer x, if $x \in S$?

- How large is the set S? (let n=|S|)
- How large are the integers?
  - integers in [0,M] (key space)
  - If M=O(n), we can use *counting sort* of linear space, and the query time is constant
  - We consider the case $M \gg n$



Information Retrieval Problem:

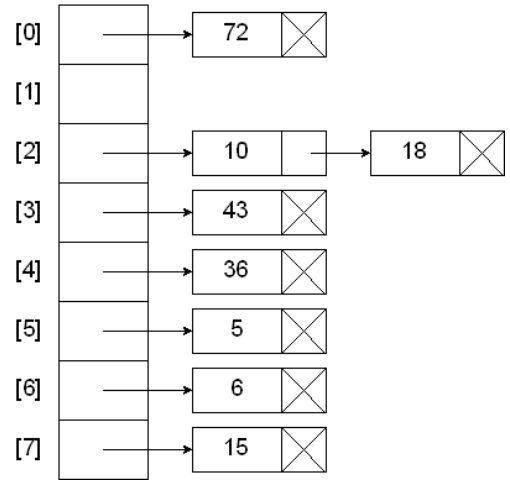
- Two commonly used schemes:
  - Sorted table & binary search:



  - Hash table

Hash key = key % table size

```
4    = 36 % 8
2    = 18 % 8
0    = 72 % 8
3    = 43 % 8
6    =  6 % 8
2    = 10 % 8
5    =  5 % 8
7    = 15 % 8
```



The query time for n elements?

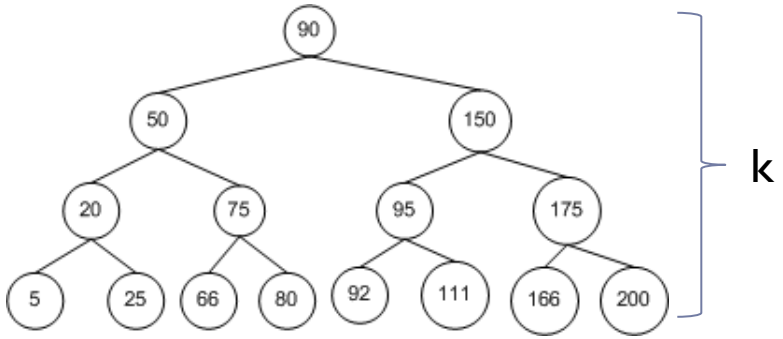


Information Retrieval Problem:

- Two commonly used schemes:

The query time for n elements?

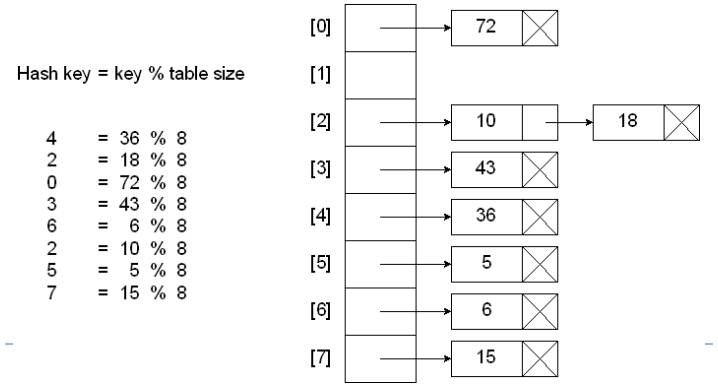
  - Sorted table & binary search:
  - What's the maximum number of n of worst-case k probes?
  - $1+2+4+\ldots+2^{k-1}=2^k-1$
  - Thus, for $2^{k-1} \le n \le 2^k-1$ we need k probes
  - $k=\lceil \log(n+1) \rceil$



  - Hash table

    on average: O(1)

    worst-case: O(n)

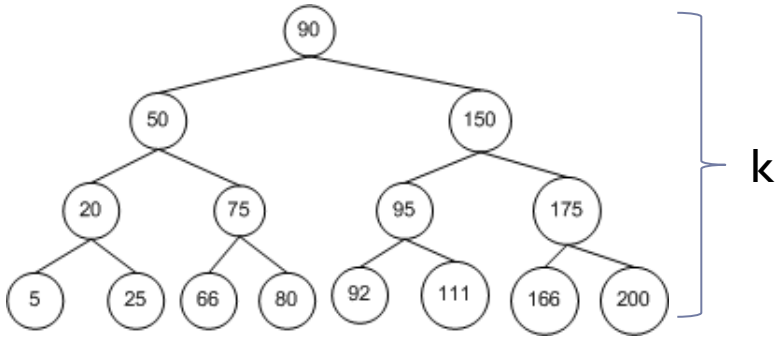




Information Retrieval Problem:

- Two commonly used schemes:
  - Sorted table & binary search:
  - What's the maximum number of n of worst-case k probes?
  - $1+2+4+\ldots+2^{k-1}=2^k-1$
  - Thus, for $2^{k-1} \le n \le 2^k-1$ we need k probes
  - $k=\lceil \log(n+1) \rceil$

The query time for n elements?



  - Hash table

    on average: O(1)

    worst-case: O(n)

Is $\lceil \log(n+1) \rceil$ optimal?
(in worst-case)



For n=2

- $\lceil \log(n+1) \rceil = 2$ (probes)
- Theorems: *[Yao 1981]*
  - for n=2, M=3, we only need 1 probe!
  - for n=2, M≥4, we need 2 probes

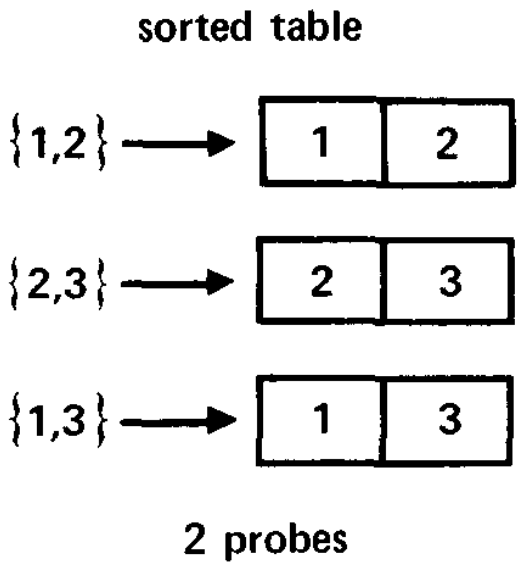


Formally stated

- Key space: {1,2,…,M}
- We are interested in storing a set S of n distinct keys into a table of size n
- A table structure $\mathcal{T}$:
  - How a particular set S be placed in the table
  - A permutation on all S such that |S|=n
  - $\mathcal{T}$: S→(s_1,…,s_n) such that s_1,…,s_n∈S
- Search Strategy $\mathcal{S}$:
  - how to find a key K in the table
- Cost($\mathcal{T}$,$\mathcal{S}$): # probes in the worst case



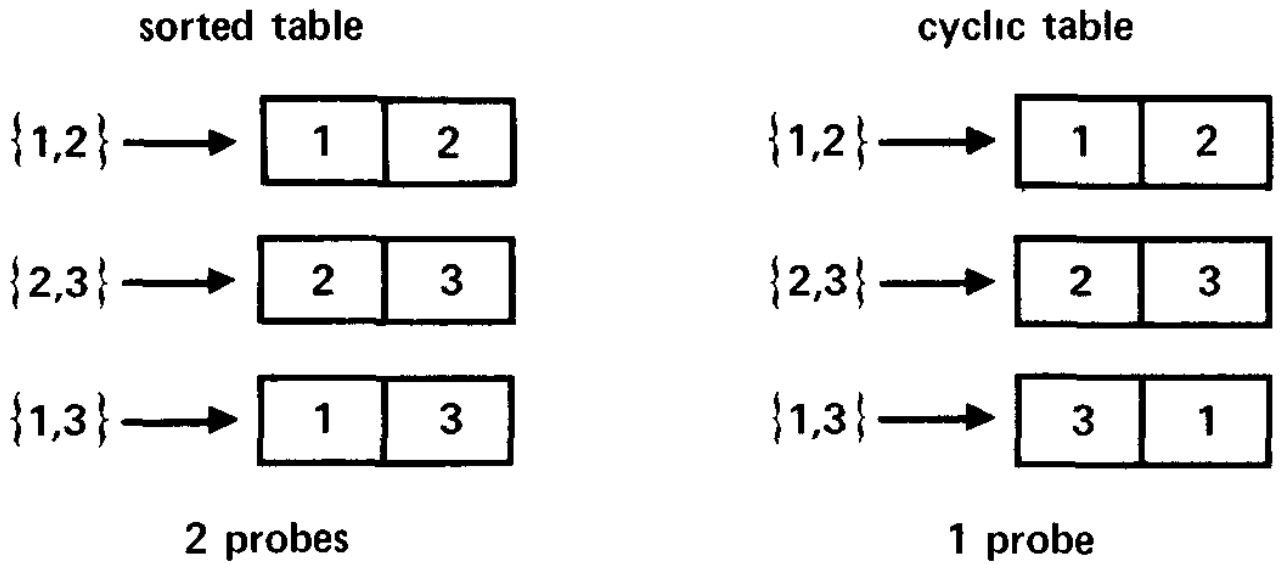
- For n=2, M=3, we need 1 probe
  - Why?



In this case, if we want to find 2, we need 2 probes

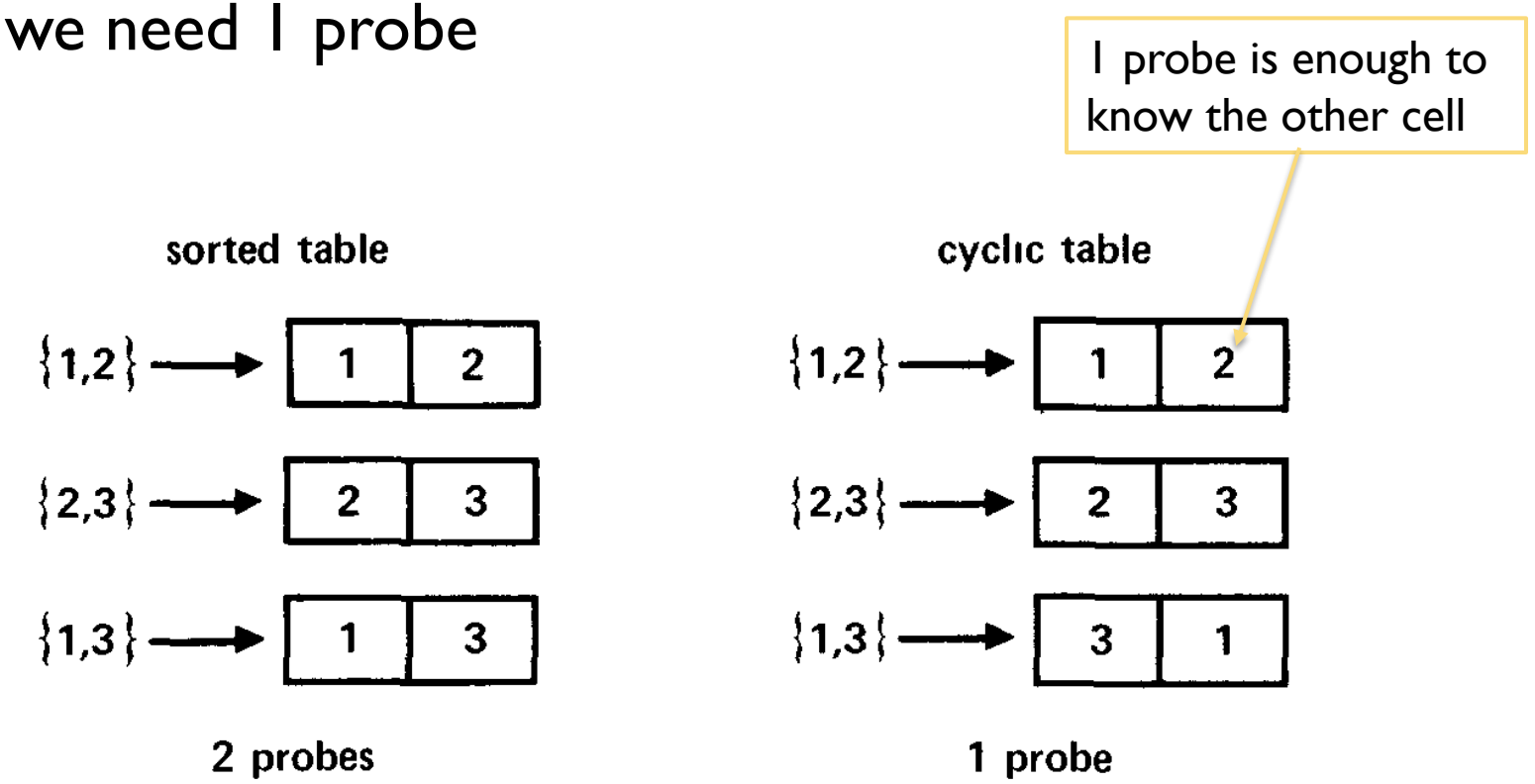


- For n=2, M=3, we need 1 probe
  - "Cyclic table"





- For n=2, M=3, we need 1 probe
  - "Cyclic table"



  - Thus, the sorted table is not optimal for n=2, M=3
  - But we can show that it is optimal for n=2, M=4



Graphs

- For n=2, any table structure $\mathcal{T}$ can be represented by a directed graph
  - draw an edge i→j if the pair {i,j} is stored as (i,j)
  - *e.g. when S={2,4}, the table stores (4,2)*

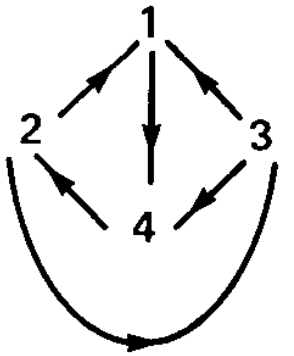


FIG 2 A typical table structure for $n = 2$, $m = 4$

Graphs

- For n=2, any table structure $\mathcal{T}$ can be represented by a directed graph
  - draw an edge i→j if the pair {i,j} is stored as (i,j)

- {1,2,4} shows a cycle
  - as in 1 probe M=3 case

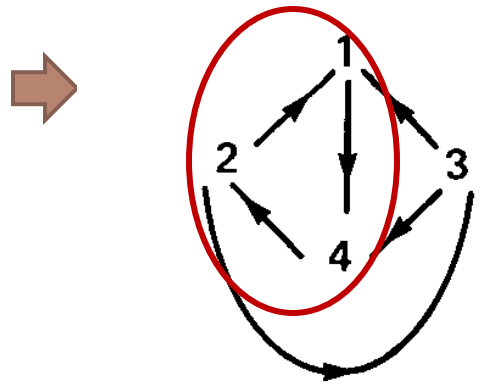


FIG 2 A typical table structure for $n = 2$, $m = 4$

Graphs

- For n=2, any table structure $\mathcal{T}$ can be represented by a directed graph
  - draw an edge i→j if the pair {i,j} is stored as (i,j).

- However, there must be 3 acyclic vertices, and we need 2 probes
  - Thus, sorted table is optimal

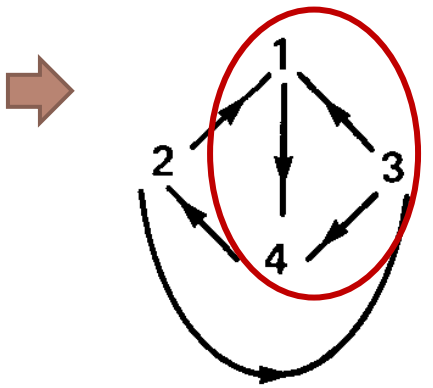


FIG 2 A typical table structure for $n = 2$, $m = 4$

Every vertex is associated with 2 in-edges or 2 out-edges



Generalized

- When $n \geq 2$ and $M \geq 2n-1$, if the table structure *always* stores keys in *a sorted order*, then $\lceil \log(n+1) \rceil$ probes are needed in worst-case for any search strategy
  - prove by induction



Generalized

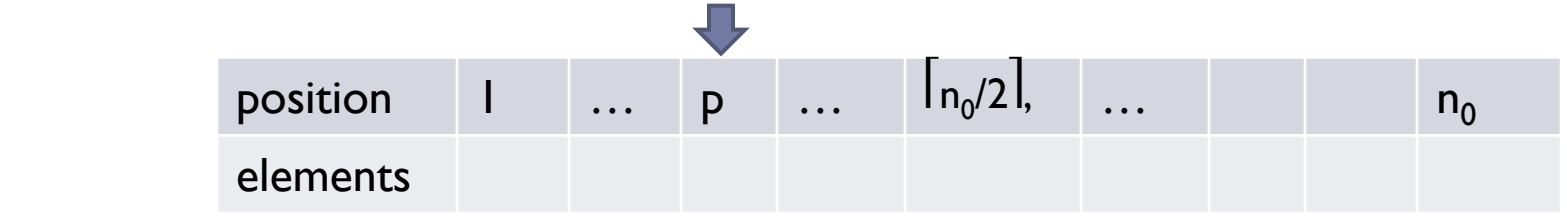
- When $n \geq 2$ and $M \geq 2n-1$, if the table structure *always* stores keys in *a sorted order*, then $\lceil \log(n+1) \rceil$ probes are needed in worst-case for any search strategy
  - (assume it is just the natural order from 1 to M)

Proof:

- Base: 2 probes are required for n=2 and M≥3 *for sorted order*
- Inductive: assume it is true for $n<n_0$, we consider $n=n_0$, $M \geq 2n_0-1$

  We want to find $K=n_0$ in the list,

  (W.l.o.g.) if the first probe is at position $p \leq \lceil n_0/2 \rceil$,



Generalized

- When $n \geq 2$ and $M \geq 2n-1$, if the table structure *always* stores keys in *a sorted order*, then $\lceil \log(n+1) \rceil$ probes are needed in worst-case for any search strategy

Proof:

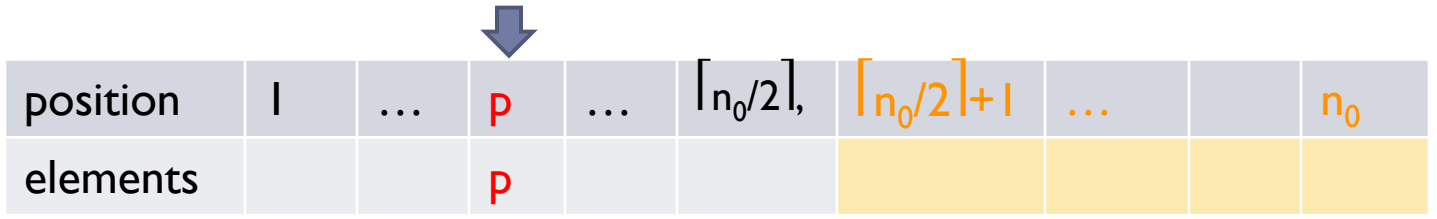
- Base: 2 probes are required for n=2 and M≥3 *for sorted order*
- Inductive: assume it is true for $n<n_0$, we consider $n=n_0$, $M \geq 2n_0-1$

  We want to find $K=n_0$ in the list,

  (W.l.o.g.) if the first probe is at position $p \leq \lceil n_0/2 \rceil$,

  T(p)=p in the worst-case (*adversary proof: someone against you!*)

  Then $T[\lceil n_0/2 \rceil+1, \ldots, n_0]$ may contain any subset of $\{\lceil n_0/2 \rceil+1, \ldots M\}$, which is sorted





- When $n \geq 2$ and $M \geq 2n-1$, if the table structure *always* stores keys in *a sorted order*, then $\lceil \log(n+1) \rceil$ probes are needed in worst-case for any search strategy.

Proof:

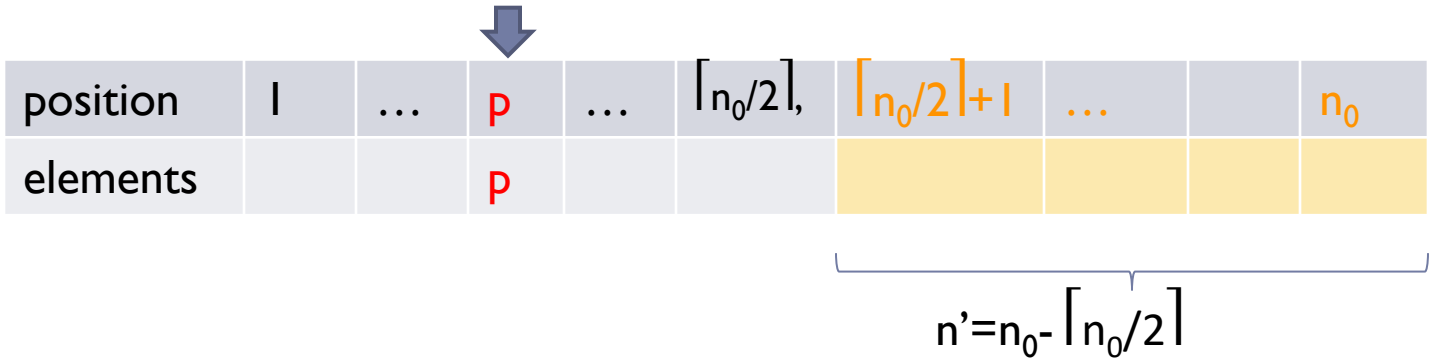
- Inductive: assume it is true for $n<n_0$, we consider $n=n_0$, $M \geq 2n_0-1$

  We want to find $K=n_0$ in the list,

  (W.l.o.g.) if the first probe is at position $p \leq \lceil n_0/2 \rceil$,

  $T(p)=p$ in the worst-case (*adversary proof: someone against you!*)

  Then $T[\lceil n_0/2 \rceil+1, \ldots, n_0]$ may contain any subset of $\{\lceil n_0/2 \rceil+1, \ldots M\}$, which is sorted *($m' \geq 2n'-1$)*



  we want to find n' in this sub-list of size n'

  By induction hypothesis, the number of probes is

$$1+\lceil \log(n'+1) \rceil = 1+\lceil \log(\lfloor n_0/2 \rfloor+1) \rceil \geq \lceil \log(n_0+1) \rceil$$



- When $n \geq 2$ and $M \geq 2n-1$, if the table structure *always* stores keys in *a sorted order*, then $\lceil \log(n+1) \rceil$ probes are needed in worst-case for any search strategy

- This also works for some fixed permutation $\sigma \in S_n$ *(Symmetric group)*
  - Such that the table structure stores $S=\{a_1<a_2<\ldots<a_n\} \subseteq [1,...,M]$ as:

$$(a_{\sigma(1)}, a_{\sigma(2)}, \ldots, a_{\sigma(n)})$$

  - (you know the rank of each position!)



Theorem 1

- For all n, there is an N(n) such that we need $\lceil \log(n+1) \rceil$ probes for $M \geq N(n)$
  - We have proved for some fixed order table structure for $M \geq 2n-1$

This also works for some fixed permutation $\sigma \in S_n$ *(Symmetric group)* such that the table structure stores $S=\{a_1<a_2<\ldots<a_n\} \subseteq [1,\ldots,M]$ as:

$$(a_{\sigma(1)}, a_{\sigma(2)}, \ldots, a_{\sigma(n)})$$

- But for any table structure?



Theorem 1

- For all n, there is an N(n) such that we need $\lceil \log(n+1) \rceil$ probes for $M \geq N(n)$
  - We have proved for some fixed order table structure for $M \geq 2n-1$

> This also works for some fixed permutation $\sigma \in S_n$ *(Symmetric group)* such that the table structure stores $S=\{a_1<a_2<\ldots<a_n\} \subseteq [1,\ldots,M]$ as:
>
> $$(a_{\sigma(1)}, a_{\sigma(2)}, \ldots, a_{\sigma(n)})$$

- For M large enough, for any table structure $\mathcal{T}$,
  - there is a subset $\Gamma \subseteq [1,\ldots,M]$ where $|\Gamma|=2n-1$
  - all tables of $S \subseteq \Gamma$ *(where $|S|=n$)* in $\mathcal{T}$ are according to a fixed permutation σ



Formally

- Let $\mathcal{A}=\{S\subseteq[1,\ldots,M]\mid |S|=n\}$
- Given a table structure $\mathcal{T}$, partition $\mathcal{A}$ into n! parts according to the permutation of tables in $\mathcal{T}$

for all σ:
$\mathcal{A}_\sigma=\{S=\{a_1<a_2<\ldots<a_n\}\in\mathcal{A} \mid S \text{ is stored as } (a_{\sigma(1)}, a_{\sigma(2)}, \ldots, a_{\sigma(n)}) \text{ in } \mathcal{T}\}$



Formally

- Let $\mathcal{A}=\{S\subseteq[1,\ldots,M]|\ |S|=n\}$
- Given a table structure $\mathcal{T}$, partition $\mathcal{A}$ into n! parts according to the permutation of tables in $\mathcal{T}$

for all σ:
$\mathcal{A}_\sigma=\{S=\{a_1<a_2<\ldots<a_n\}\in\mathcal{A}$ | S is stored as $(a_{\sigma(1)}, a_{\sigma(2)}, \ldots, a_{\sigma(n)})$ in $\mathcal{T}\}$

- When M large enough, there is a subset $\Gamma\subseteq[1\ldots M]$ and $|\Gamma|=2n-1$, such that
  $\exists\sigma$,$\forall A\subseteq\Gamma$ and $|A|=n$, $A\in\mathcal{A}_\sigma$

By Ramsey theorem



Ramsey theorem—Simplest Case

- For any 6 people, there exists 3 people who know each other or 3 people who don't know each other
  - acquaintance are mutual: undirected graph
  - how to prove?



Ramsey theorem—Simplest Case

- For any 6 people, there exists 3 people who know each other or 3 people who don't know each other
  - acquaintance are mutual: undirected graph
  - how to prove?

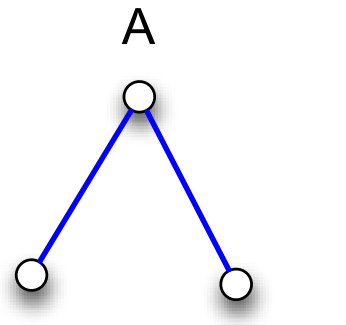
- For any 9 people, there exists 3 people who know each other or 4 people who don't know each other



Theorem on friends and strangers

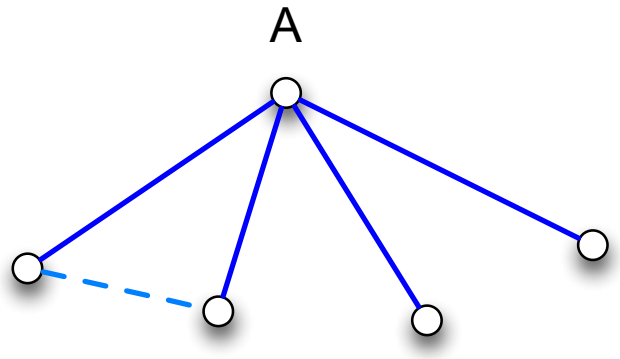
- For any 9 people, there exists 3 people who know each other or 4 people who don't know each other

A knows ≤2 people



6 people case

A knows ≥4 people



The only case left is all 9 people knows 3 other people, but it's impossible since the total degree must be even



Ramsey theorem

- In combinatorics, Ramsey's theorem states that one will find monochromatic cliques in any edge labeling (with colors) of a sufficiently large complete graph

- Theorem on friends and strangers is the two color case
  - e=(A, B) is red when A knows B, blue otherwise



Ramsey theorem

- In combinatorics, Ramsey's theorem states that one will find monochromatic cliques in any edge labelling (with colors) of a sufficiently large complete graph
  - Theorem on friends and strangers is the two color case
    - e=(A, B) is red when A knows B, blue otherwise

Ramsey number: R(r, s)

Smallest number n such that there exists red cliques of size r or blue cliques of size s in K_n

R(r,s)=R(s,r)



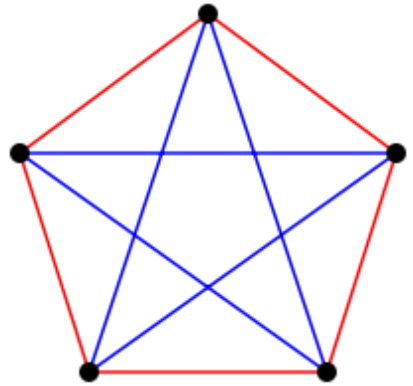
Ramsey theorem

Ramsey number: R(r, s)

Smallest number n such that there exists red cliques of size r or blue cliques of size s in K_n

R(r,s)=R(s,r)

- R(3,3)=6
  - We have shown R(3,3)≤6
  - R(3,3)>5:



Ramsey theorem

- *R(r, s) exists for all r, s*
- Proof:
  - Induction on r+s
  - R(1,n)=R(n,1)=1
  - we want to prove $R(r, s) \leq R(r-1, s)+R(r, s-1)$



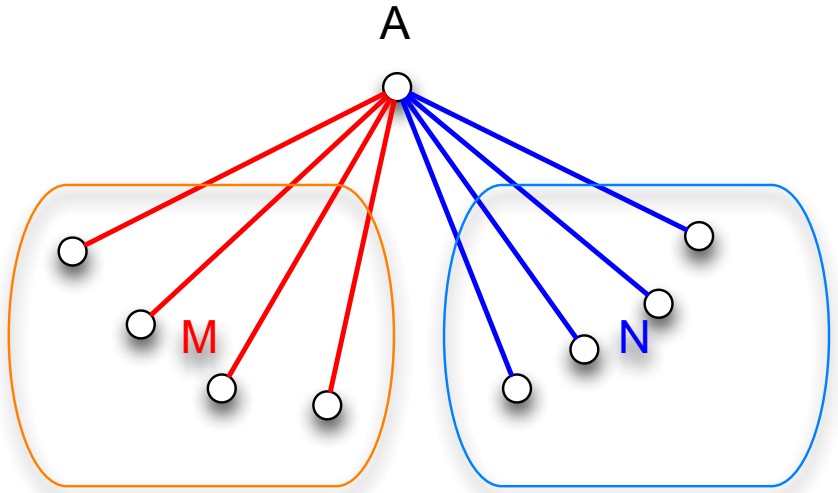
Ramsey theorem

- *R(r, s) exists for all r, s*
- Proof:
  - Induction on r+s
  - R(1,n)=R(n,1)=1
  - we want to prove $R(r, s) \leq R(r-1, s)+R(r, s-1)$
  - On complete graph with R(r-1, s)+R(r, s-1) vertices,

M: #vertices with red edges to A
N: #vertices with blue edges to A

if M+N+1= R(r-1, s)+R(r, s-1)
either M≥ R(r-1, s) or N≥R(r, s-1)

Either case is true!



- Ramsey numbers: computable, but extremely hard to compute!
  - R(3,3)=6
  - R(3,4)=9
  - R(4,4)=18
  - R(5,5)=?



- Ramsey numbers: computable, but extremely hard to compute!
  - R(3,3)=6
  - R(3,4)=9
  - R(4,4)=18

Checking all complete graphs of k nodes: $2^{\binom{k}{2}}$
(No hope for brute-force)

- WE ONLY KNOW 43 *[Exoo 1989]* ≤ R(5,5) ≤ 46 *[Angeltveit&McKay 2024]*
- $R(k,k) \leq (4-\varepsilon)^k$ $(\varepsilon=2^{-7})$ *[Campos et al. 2023]*



Ramsey numbers: computable, but extremely hard to compute!

- What we know:

r \ s	1	2	3	4	5	6	7	8	9	10
1	**1**	1	1	1	1	1	1	1	1	1
2		**2**	3	4	5	6	7	8	9	10
3			**6**	9	14	18	23	28	36	40–41[14]
4				**18**	25[9]	36–40	49–58	59[15]–79	73–105	92–135
5					**43–46**[11]	59[16]–85	80–133	101–193	133–282	149[15]–381
6						**102–160**	115[15]–270	134[15]–423	183–651	204–944
7							**205–492**	219–832	252–1368	292–2119
8								**282–1518**	329–2662	343–4402
9									**565–4956**	581–8675
10										**798–16064**



Generalization to multi-color case

- In combinatorics, Ramsey's theorem states that one will find monochromatic cliques in any edge labelling (with colors) of a sufficiently large complete graph
  - e.g. R(3,3,3)=17
  - for any finite number of colors



Back to our statement

- Let $\mathcal{A}=\{S\subseteq[1,\ldots,M]|\ |S|=n\}$
- Given a table structure $\mathcal{T}$, partition $\mathcal{A}$ into n! parts according to the permutation of tables in $\mathcal{T}$

for all σ:
$\mathcal{A}_\sigma=\{S=\{a_1<a_2<\ldots<a_n\}\in\mathcal{A}$ | S is stored as $(a_{\sigma(1)}, a_{\sigma(2)}, \ldots, a_{\sigma(n)})$ in $\mathcal{T}\}$

- When M large enough, there is a subset $\Gamma\subseteq[1\ldots M]$ and $|\Gamma|=2n-1$, such that
  $\exists\sigma$,$\forall A\subseteq\Gamma$ and $|A|=n$, $A\in\mathcal{A}_\sigma$

By Ramsey theorem extension to hypergraphs



m-Hypergraph

- A m-**hypergraph** is a generalization of a graph in which an edge can connect m vertices
  - *complete m-hypergraph*: any subset $S \subseteq V$ for $|S|=m$ is an edge
  - Ramsey theorem for m-hypergraphs:

> for any integers *m* and *c*, and any integers $n_1, \ldots, n_c$, there is an integer $R(n_1, \ldots, n_c; c, m)$ such that if the hyperedges of a complete *m*-hypergraph of order $R(n_1, \ldots, n_c; c, m)$ are colored with *c* different colors, then for some *i* between 1 and *c*, the hypergraph must contain a complete sub-*m*-hypergraph of order n_i whose hyperedges are all color *i*.



Back to our statement

- Let $\mathcal{A}=\{S\subseteq[1,\ldots,M]\mid |S|=n\}$
- Given a table structure $\mathcal{T}$, partition $\mathcal{A}$ into n! parts according to the permutation of tables in $\mathcal{T}$

for all σ:
$\mathcal{A}_\sigma=\{S=\{a_1<a_2<\ldots<a_n\}\in\mathcal{A} \mid S \text{ is stored as } (a_{\sigma(1)}, a_{\sigma(2)}, \ldots, a_{\sigma(n)}) \text{ in } \mathcal{T}\}$

- When M large enough, there is a subset $\Gamma\subseteq[1\ldots M]$ and $|\Gamma|=2n-1$, such that

  $\exists\sigma$,$\forall A\subseteq\Gamma$ and $|A|=n$, $A\in\mathcal{A}_\sigma$

Here we consider n-hypergraphs, #colors: n!
there exists a complete sub-n-hypergraph of order 2n-1 with all edges of the same color



Conclusion

- For all n, there is an N(n) such that we need $\lceil \log(n+1) \rceil$ probes for $M \geq N(n)$
  - we can also prove it with "pointers" and duplicated keys



Logarithmic Lower Bounds

- *[Pătraşcu, Demaine 2004]:*
  - Dynamic partial sum problem
  - Dynamic connectivity



Dynamic Partial Sum

- Maintain $A[1..n]$ under:
  - update(k, Δ) — modify $A[k] = A[k] + \Delta$
  - sum(k) — report $A[1] + \ldots + A[k]$

  - **Motivation**
    - range query (1D)
    - many applications in the literature
      list indexing, dynamic ranking [Dietz 89]
      dynamic arrays [Raman et al 2001]
      arithmetic coding [Fenwick 94]
      …
    - playground for lower bound techniques (many results)



General models: RAM, cell probe

- operations in a group $\mathbb{G}$
  - Here we consider $|\mathbb{G}|=2^{\delta}$, that is, elements in $\mathbb{G}$ needs δ bits
- word size for computer: $w \geq \lg n$
  - Handle w-bit data in one instruction
- update parameter Δ limited to δ bits

  natural in applications

  all previous studies considered this



Results

- [*Mihai Pătraşcu, Erik Demaine*] Any data structure requires an average running time of $\Omega\left(\frac{\delta}{w} \cdot \log n\right)$ per updates and queries
  - n: size of the array `A[1..n]`
  - $\delta = \log_2 |\mathbb{G}|$
  - w-bit words machine

- Proving lower bound: design *a hard instance* and analyze its information transfer



Partial Sum—Upper bound

- (Generalized)
- Initially: A[1…n]=(0,…,0)
- Update: update(k, Δ) modify A[k] = A[k] + Δ
- Query: Given 1≤i,j≤n, find the sum of elements in A[i…j]





Partial Sum—Upper bound

- (Generalized)
- Initially: A[1…n]=(0,…,0)
- Update: update(k, Δ) modify A[k] = A[k] + Δ
- Query: Given 1≤i,j≤n, find the sum of elements in A[i…j]

- Example:

  A[1…10]=(8, 7, 3, 20, 2, 17, 5, 21, 11, 12)

  ∑ A[6…9]=17+5+21+11=54



Good structures for Partial Sum?

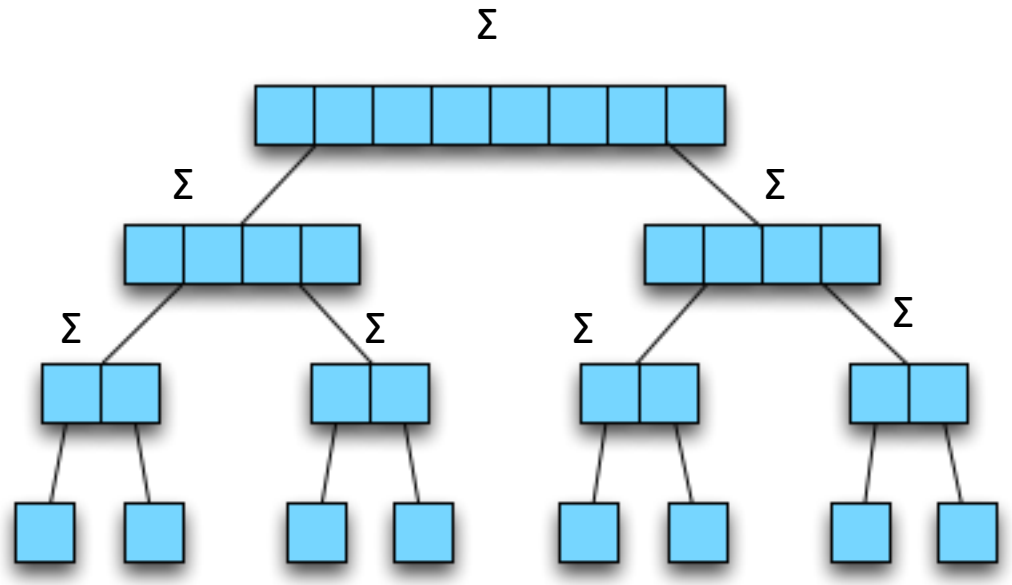
- Space: O(n)
- Update time: O(log n)
- Query time: O(log n)

- (*when* δ=*w*)



Good structures for Partial Sum?

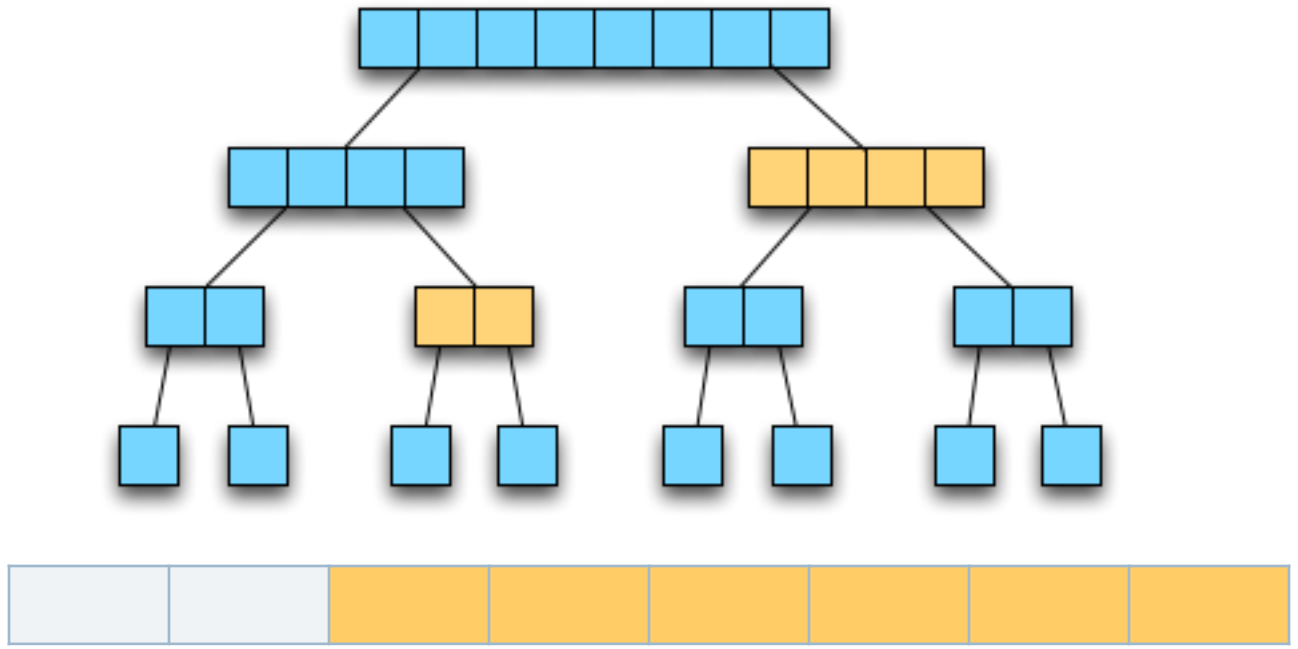
- "Binary tree":
  - Preprocess: find the summation in every following range:



Range: $[(k-1)2^i+1,\ldots,k2^i]$ of length 2^i

Good structures for Partial Sum?

- "Binary tree":
  - Preprocess: find the summation in every following range:
  - Query: every range A[i,j] can be partitioned into O(log n) such ranges

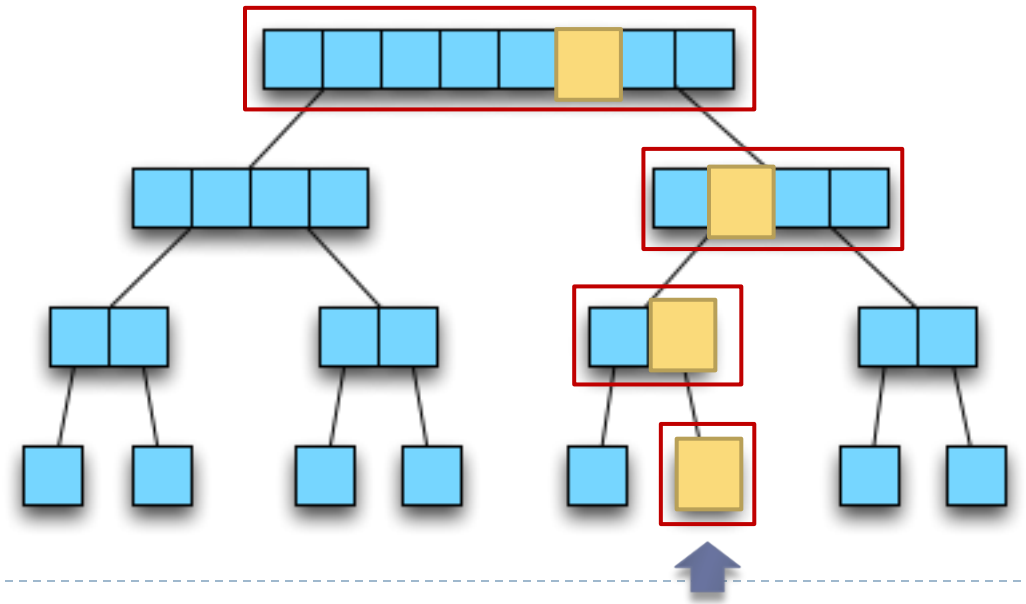




Good structures for Partial Sum?

- "Binary tree":
  - Preprocess: find the summation in every following range:
  - Query: every range A[i,j] can be partitioned into O(log n) such ranges
  - Update: O(log n) cell writes

  - *(It works for any range A[i,j])*





Lower bound

- We only consider the more restricted case:
  - Initially: A[1…n]=(0,…,0)
  - Update(k, Δ) modify A[k] = A[k] + Δ
  - Sum(k) report A[1] + … + A[k]



Lower bound

- We only consider the more restricted case:
  - Initially: A[1…n]=(0,…,0)
  - Update(k, Δ) modify A[k] = A[k] + Δ
  - Sum(k) report A[1] + … + A[k]
  - *(Consider RAM on w-bit words, w≥log n, elements of A[i] and Δ are δ-bit integers)*

- We design a "bad" sequence:
  - Choose a random permutation $\pi \in S_n$
  - Choose a uniformly random sequence $\langle \Delta_1, \ldots, \Delta_n \rangle \in \mathbb{G}^n$

S_n: symmetric group of size n

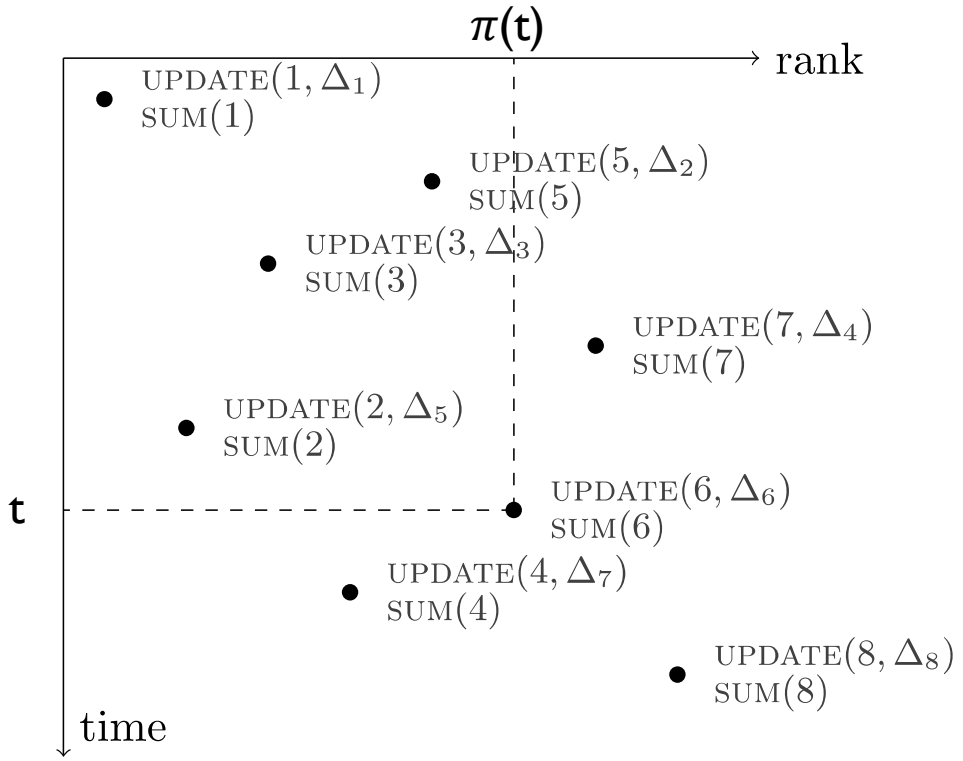
```
For t←1 to n                          \\ time t
        Update(π(t), Δ_t);
        Sum(π(t))
```



Example

```
For t←1 to n                    \\ time t
        Update(π(t), Δ_t);
        Sum(π(t))
```

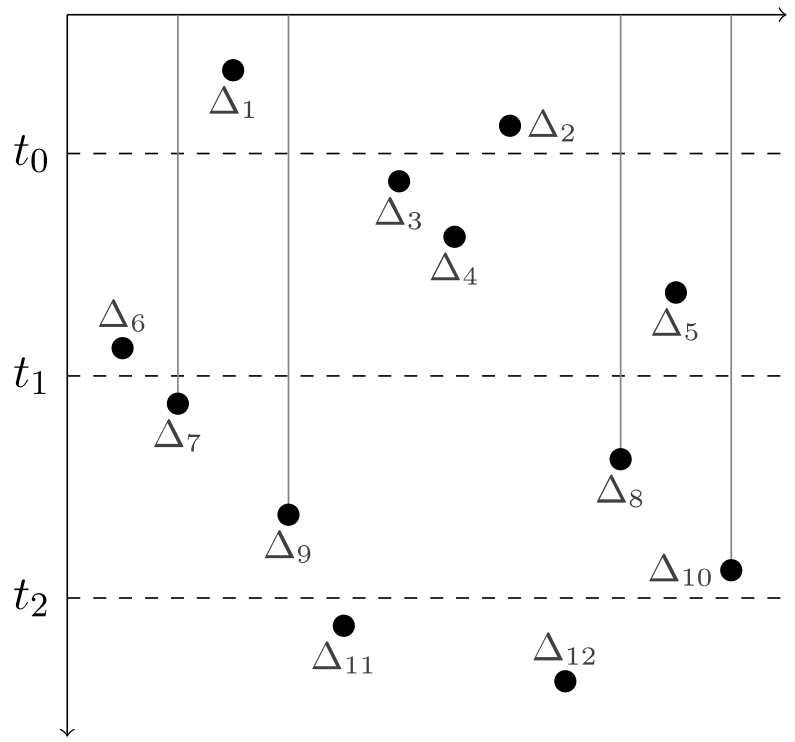
- If n=8 and π=(1,5,3,7,2,6,4,8),
- Sum(π(t)): *(sum of A[1] + … + A[π(t)] at time t)*
  - the sum of updates in the rectangle [0,t] × [0, π(t)]





Time interval

- Given time intervals $[t_0,t_1]$ and $[t_1,t_2]$,
  - How much information needed to be transferred from $[t_0,t_1]$ to $[t_1,t_2]$





Time interval

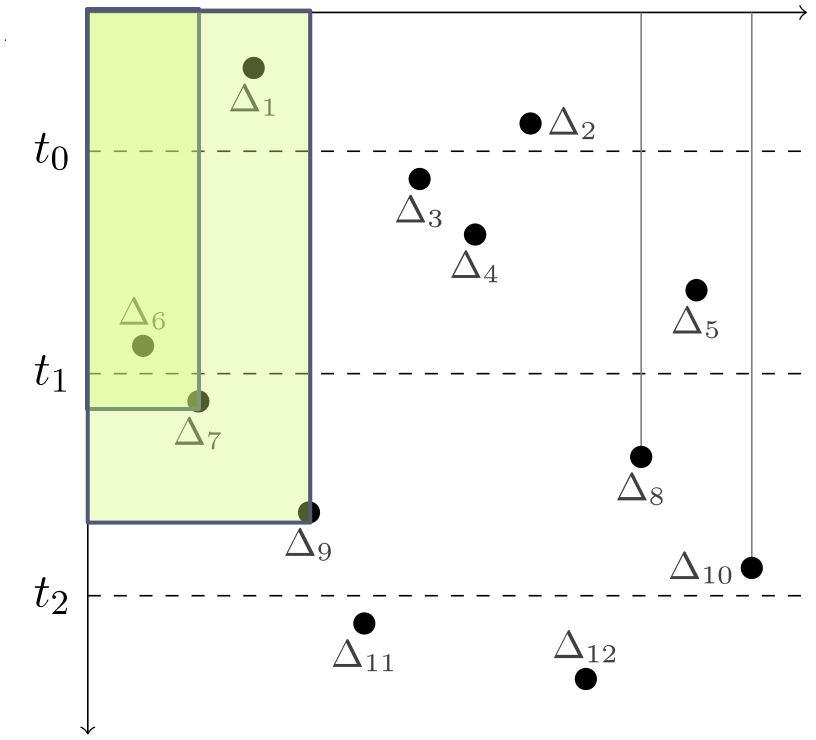
- Given time intervals $[t_0,t_1]$ and $[t_1,t_2]$,
  - How much information needed to be transferred?

The queries in $[t_1,t_2]$ needed to know about $[t_0,t_1]$:

Δ_6;
$\Delta_6 + \Delta_3 + \Delta_4$;
$\Delta_6 + \Delta_3 + \Delta_4 + \Delta_5$;

equivalent to:

Δ_6;
$\Delta_3 + \Delta_4$;
Δ_5;





Time interval

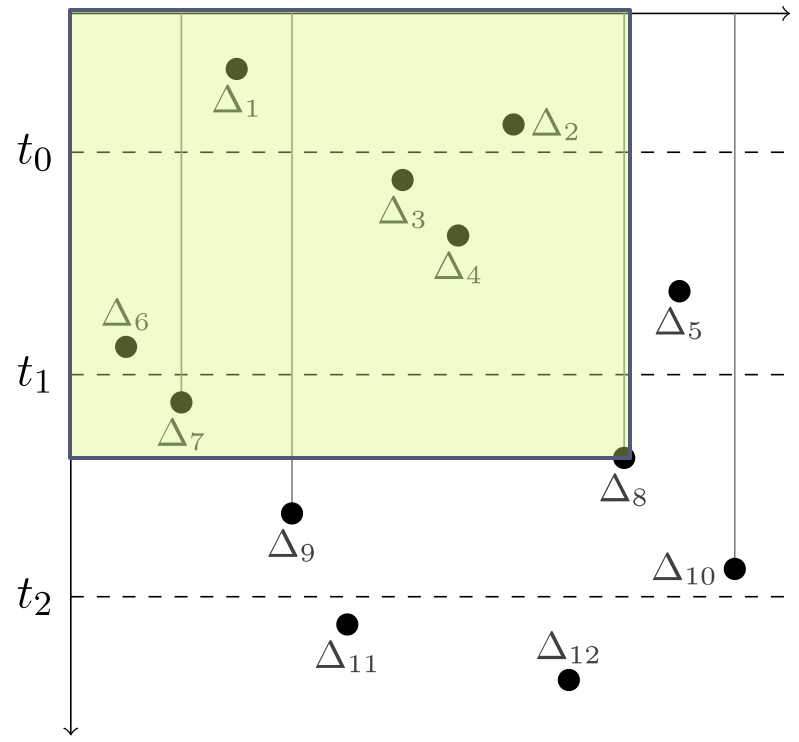
- Given time intervals $[t_0,t_1]$ and $[t_1,t_2]$,
  - How much information needed to be transferred?

The queries in $[t_1,t_2]$ needed to know about $[t_0,t_1]$:

Δ_6;
$\Delta_6 + \Delta_3 + \Delta_4$;
$\Delta_6 + \Delta_3 + \Delta_4 + \Delta_5$;

equivalent to:

Δ_6;
$\Delta_3 + \Delta_4$;
Δ_5;





Time interval

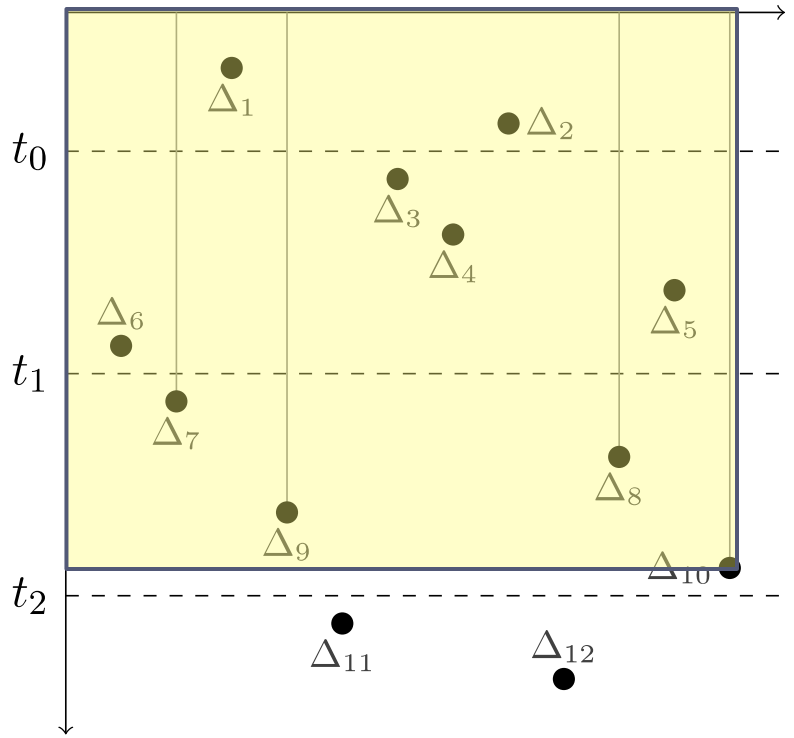
- Given time intervals $[t_0,t_1]$ and $[t_1,t_2]$,
  - How much information needed to be transferred?

The queries in $[t_1,t_2]$ needed to know about $[t_0,t_1]$:

Δ_6;
$\Delta_6+\Delta_3+\Delta_4$;
$\Delta_6+\Delta_3+\Delta_4+\Delta_5$;

equivalent to:

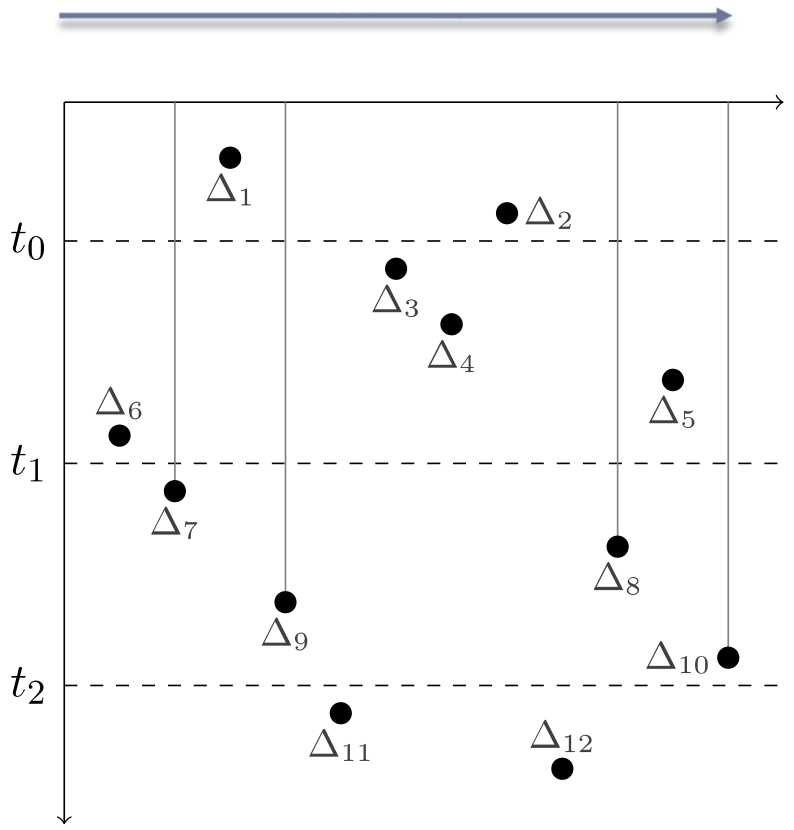
Δ_6;
$\Delta_3+\Delta_4$;
Δ_5;



Three independent random variables

Interleave information

- $IL(t_0, t_1, t_2)$:
  - We sort the set $\{\pi(t_0),\ldots,\pi(t_2)\}$
  - $IL(t_0, t_1, t_2)$ is the number of transitions between a value $\pi(i)$ with $i<t_1$, and a consecutive value $\pi(j)$ with $j>t_1$





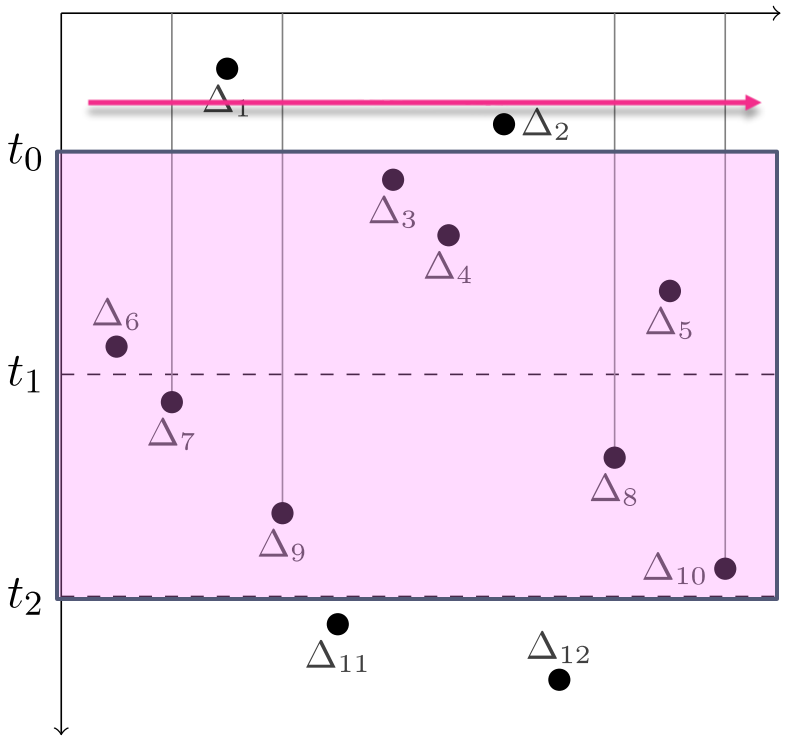
Interleave information

- $IL(t_0, t_1, t_2)$:
  - We sort the set $\{\pi(t_0),\ldots,\pi(t_2)\}$
  - $IL(t_0, t_1, t_2)$ is the number of transitions between a value $\pi(i)$ with $i<t_1$, and a consecutive value $\pi(j)$ with $j>t_1$

- In this example:
  - $t_0=3; t_1=6.5; t_2=10$
  - sort $\pi(3)$ to $\pi(10)$:

$(\pi(6), \pi(7), \pi(9), \pi(3), \pi(4), \pi(8), \pi(5), \pi(10))$

  - what's $IL(t_0, t_1, t_2)$?





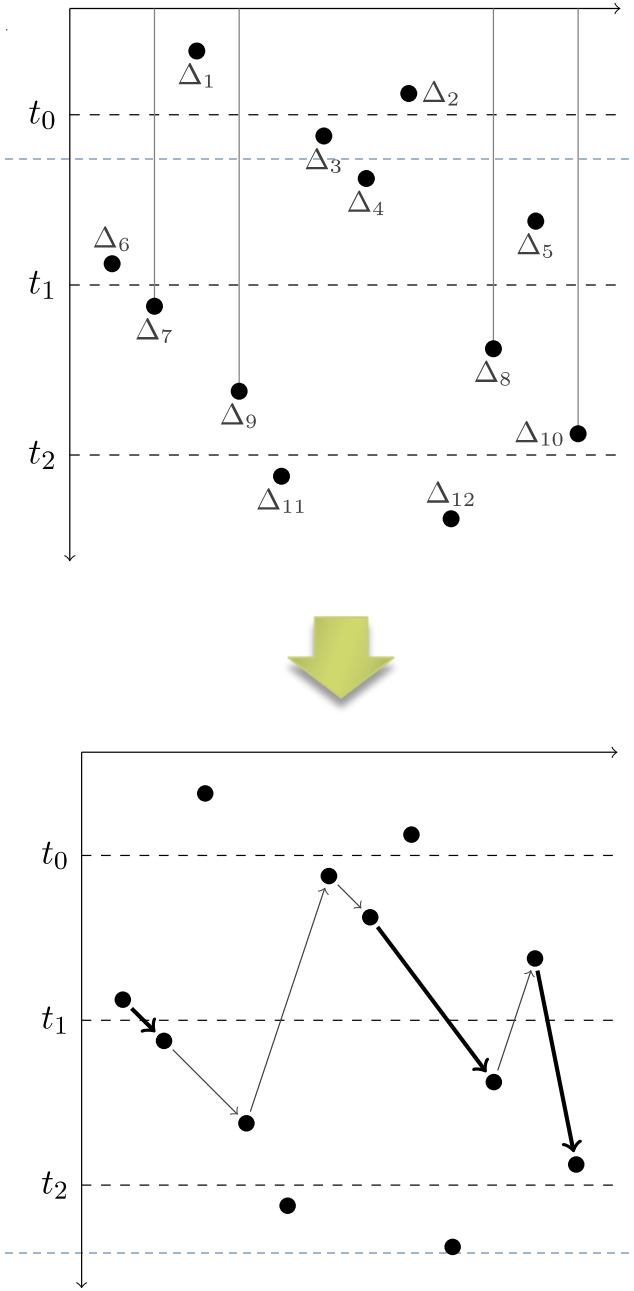
Interleave information

- $IL(t_0, t_1, t_2)$:
  - We sort the set $\{\pi(t_0),\ldots,\pi(t_2)\}$
  - $IL(t_0, t_1, t_2)$ is the number of transitions between a value $\pi(i)$ with $i<t_1$, and a consecutive value $\pi(j)$ with $j>t_1$

- In this example:
  - $t_0=3; t_1=6.5; t_2=10$
  - sort $\pi(3)$ to $\pi(10)$:

$(\pi(6), \pi(7), \pi(9), \pi(3), \pi(4), \pi(8), \pi(5), \pi(10))$

  - $IL(t_0, t_1, t_2)=3$

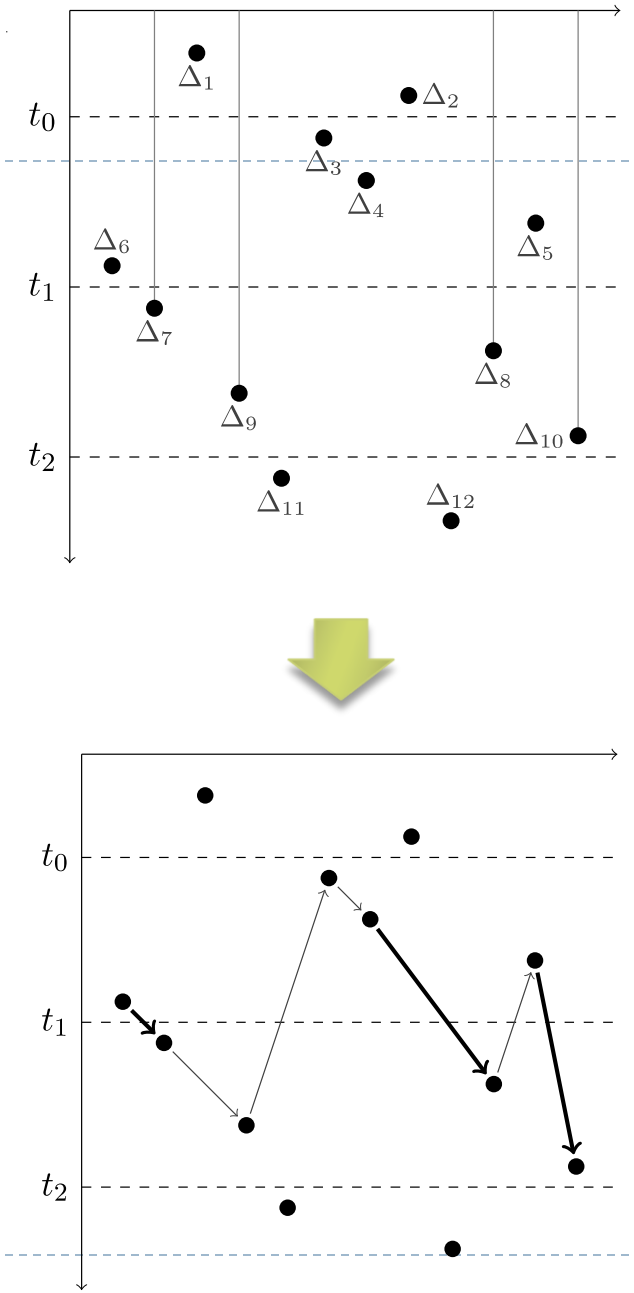


Interleave information

- $IL(t_0, t_1, t_2)$:
  - We sort the set $\{\pi(t_0),\ldots,\pi(t_2)\}$
  - $IL(t_0, t_1, t_2)$ is the number of transitions between a value $\pi(i)$ with $i<t_1$, and a consecutive value $\pi(j)$ with $j>t_1$
  - In this example, $IL(t_0, t_1, t_2)=3$

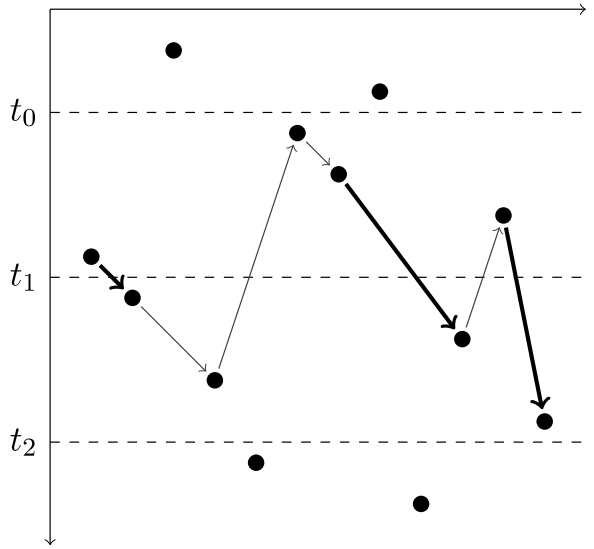
The queries in $[t_1,t_2]$ need to know 3 independent variables: Δ_6, $\Delta_3+\Delta_4$, Δ_5;

  - *Thus, $IL(t_0, t_1, t_2)$ describe the number of information needed to transfer from $[t_0,t_1]$ to $[t_1,t_2]$*



Expectation of $IL(t_0, t_1, t_2)$

- We want to compute $E[IL(t_0, t_1, t_2)]$:
  - only consider $t_2-t_1=t_1-t_0$ *(suppose there are 2k points, k each side)*

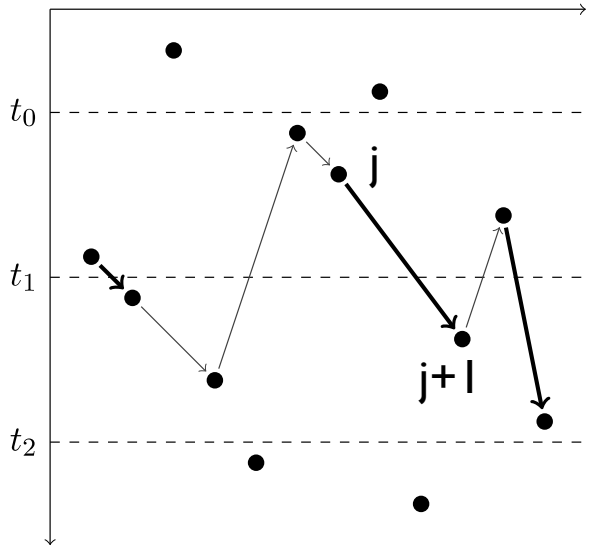




Expectation of $IL(t_0, t_1, t_2)$

- We want to compute $E[IL(t_0, t_1, t_2)]$:
  - only consider $t_2-t_1=t_1-t_0$ *(suppose there are 2k points, k each side)*
  - When we sort the random list $\{\pi(1),\ldots,\pi(2k)\}$
  - Pr[j is in $[t_0, t_1]$]=k/(2k)=1/2
  - Pr[j+1 is in $[t_1, t_2]$ | j is in $[t_0, t_1]$]=k/(2k-1)
  - Due to the linearity of expectation:
  - $E[IL(t_0, t_1, t_2)]=(2k-1)\cdot\frac{k}{2(2k-1)}=\frac{k}{2}$

    (where $t_2-t_1=t_1-t_0$)



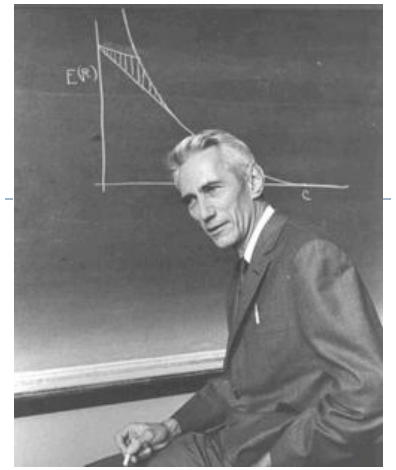


Entropy

Suppose we have a single event A with possible outcomes $\{a_i\}$, each with prob. $\{p_i\}$

Then it's entropy is:

$$H = -\sum_i p_i \log p_i$$



Claude Shannon 1916 - 2001

If A has 2^k possible outcomes, each with prob. $1/2^k$, then its entropy is just k

The entropy is the lower bound of expected bit-length of any encodings of A



- We know that the "entropy" of k independent fair coins is k

- Let Δ^* be the updates not in the range $[t_0,t_1]$

  $$H(\text{answers in } [t_1,t_2] \mid \Delta^*) = \delta \cdot IL(t_0, t_1, t_2)$$

  - since the answers depend on $IL(t_0, t_1, t_2)$ independent random variables uniformly chosen from $\mathbb{G}$, and $\log|\mathbb{G}|=\delta$



Information transfer IT(t_0, t_1, t_2)

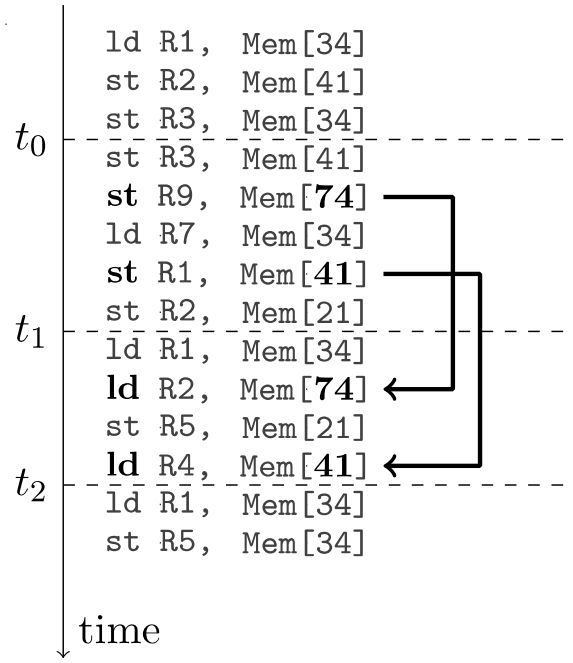
- IT(t_0, t_1, t_2) is the set of memory locations which:
  - were read at a time $t_r \in [t_1, t_2]$
  - were written at a time $t_w \in [t_0, t_1]$, and *not overwritten* during $[t_w + 1, t_r]$



Information transfer IT(t_0, t_1, t_2)

- IT(t_0, t_1, t_2) is the set of memory locations which:
  - were read at a time $t_r \in [t_1, t_2]$
  - were written at a time $t_w \in [t_0, t_1]$, and *not overwritten* during $[t_w + 1, t_r]$

st: store, ld: load





- Note that $IL(t_0, t_1, t_2)$ only depends on the permutation π
- $IT(t_0, t_1, t_2)$ may also depend on the data set Δ
- We can prove:

$$E_\Delta[|IT(t_0, t_1, t_2)|] > \frac{\delta}{2w} \cdot IL(t_0, t_1, t_2) - 1$$

  - original paper proves this by entropy



- Prove: $H(\text{answers in } [t_1,t_2] \mid \Delta^*) \le w + 2w \cdot E_\Delta[|IT(t_0, t_1, t_2)|]$

  - The entropy is upper bounded by the average length of any encoding
  - the encoding here is just the information transfer:

    the cardinality $|IT(t_0, t_1, t_2)|$

    the address of each cell in $IT(t_0, t_1, t_2)$; an address is at most w bits in our model

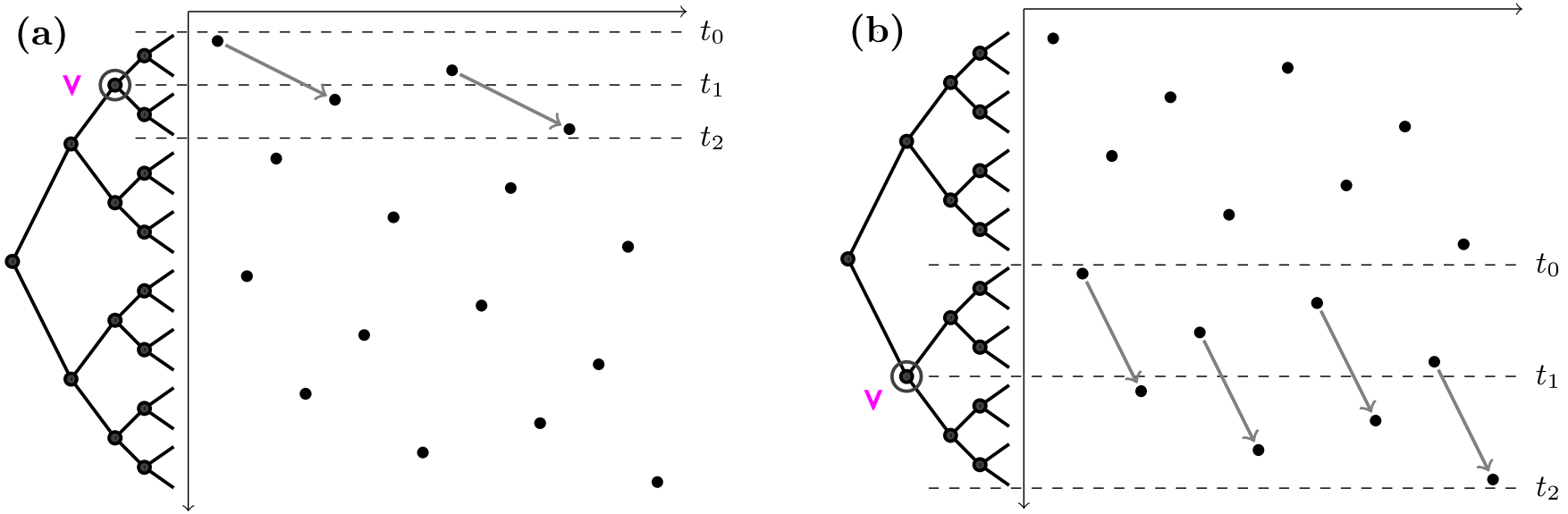
    the contents of each cell at time t_1, which takes w bits per cell

- Since $H(\text{answers in } [t_1,t_2] \mid \Delta^*) = \delta \cdot IL(t_0, t_1, t_2)$, we have:

$$E_\Delta[|IT(t_0, t_1, t_2)|] > \frac{\delta}{2w} \cdot IL(t_0, t_1, t_2) - 1$$

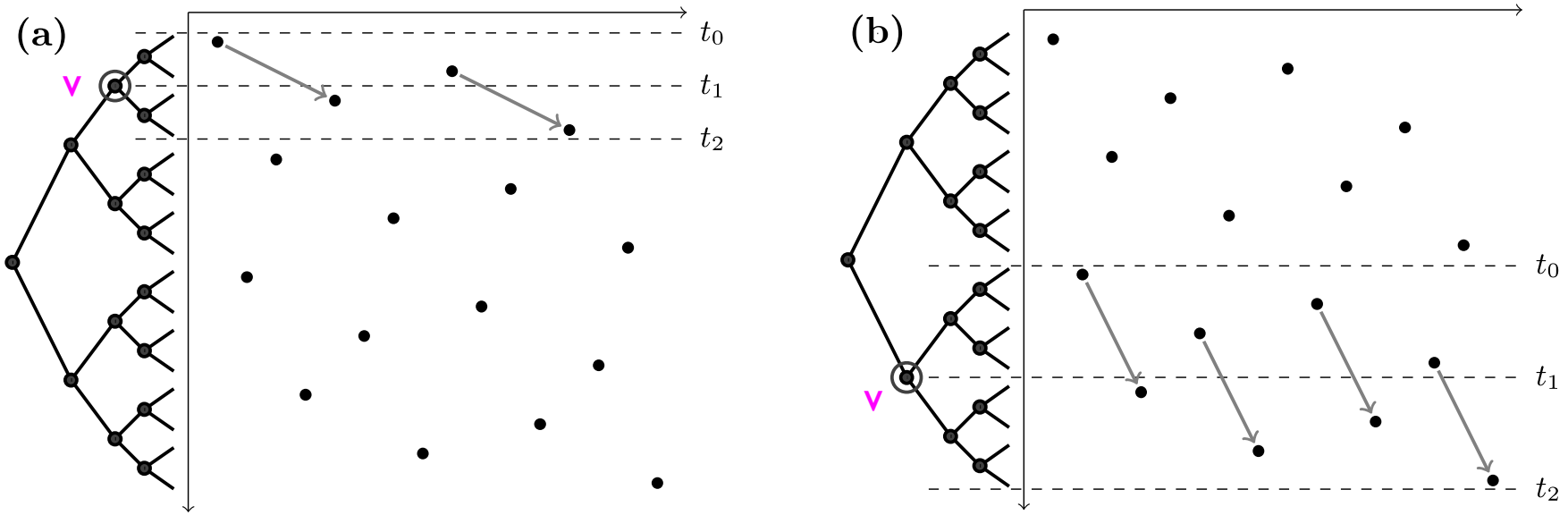


- So to bound the total IT, we need to bound IL
- As before, we can use a binary partition of the whole procedure of length n (=2^i):





- So to bound the total IT, we need to bound IL
- As before, we can use a binary partition of the whole procedure of length n (=2^i):



- As shown in this example,
  - IT(v) and IL(v) are defined as IT(t_0, t_1, t_2) and IL(t_0, t_1, t_2) where [t_0,t_1] and [t_1,t_2] are represented by two children of v



- So to bound the total IT, we need to bound IL
- As before, we can use a binary partition of the whole procedure of length n (=2^i):

- We only need to bound $\Sigma_v E[|IT(v)|]$, as the "read/write" operation needs to perform $\Sigma_v |IT(v)|$ times

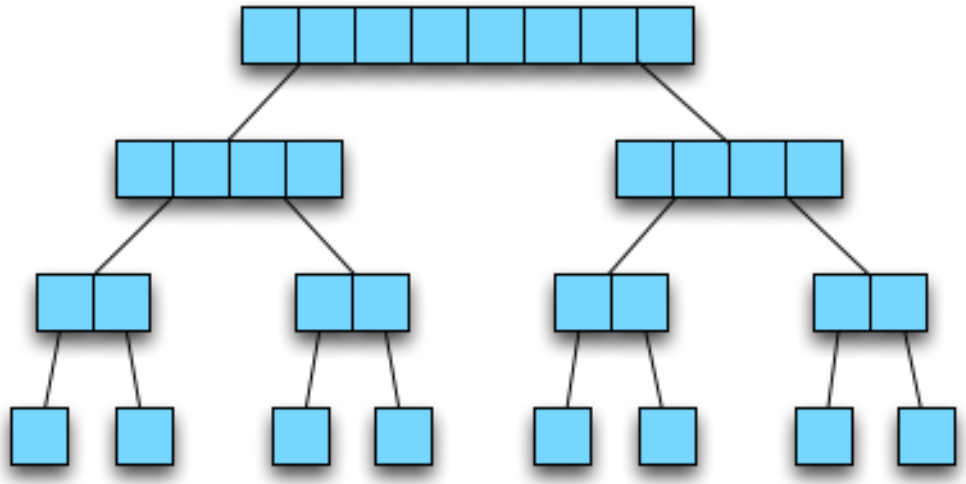
  - $\Sigma_v E[|IT(v)|] > \frac{\delta}{2w} \cdot \Sigma_v IL(v) - n$, since $E[|IT(t_0, t_1, t_2)|] > \frac{\delta}{2w} \cdot IL(t_0, t_1, t_2) - 1$



- Remember that $E[IL(t_0, t_1, t_2)]=(2k-1)\cdot\frac{k}{2(2k-1)}=\frac{k}{2}$

  (where $t_2-t_1=t_1-t_0=k$)

- So $E_\pi[\Sigma_v IL(v)]\approx\frac{1}{4}n\log n$, thus $E[\Sigma_v|IT(v)|]=\Theta\left(\frac{\delta}{w}\cdot n\log n\right)$
  - Every operation needs $\Omega\left(\frac{\delta}{w}\cdot\log n\right)$ cell probes on average





- "Logarithmic Lower Bounds in the Cell-Probe Model"

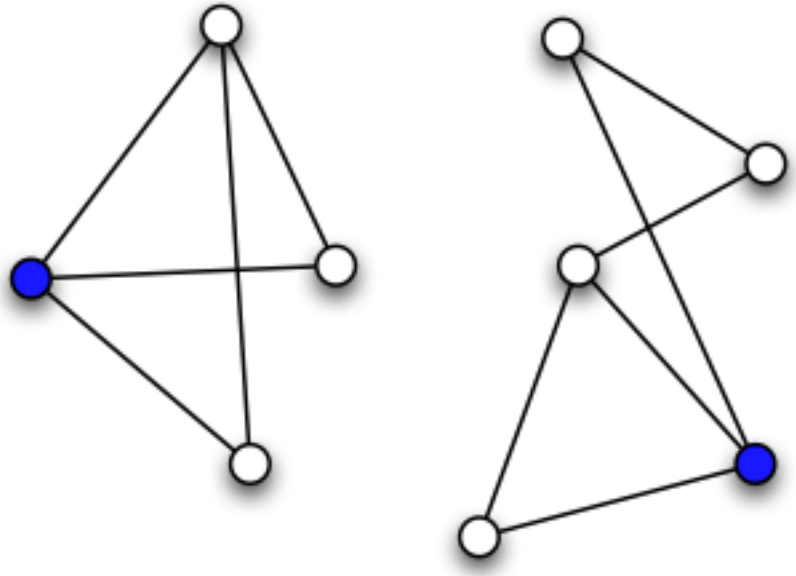
  By Mihai Pâtraşcu and Erik Demaine

- Gives $\Omega(\log n)$ lower bounds on:
  - Partial sum
  - Dynamic connectivity (by reduction from partial sum)
  - …



Dynamic Connectivity

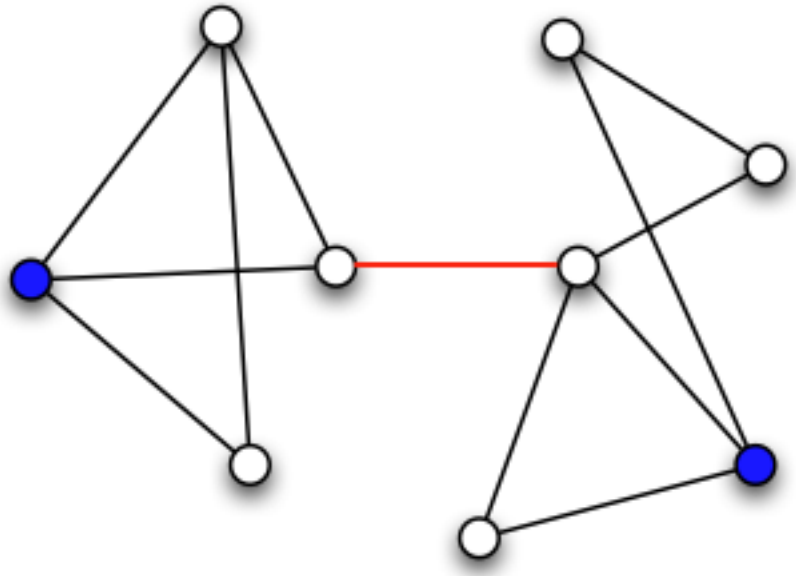
- Connectivity problem: in an undirected graph G, judge whether any two vertices are connected by a path





Dynamic Connectivity

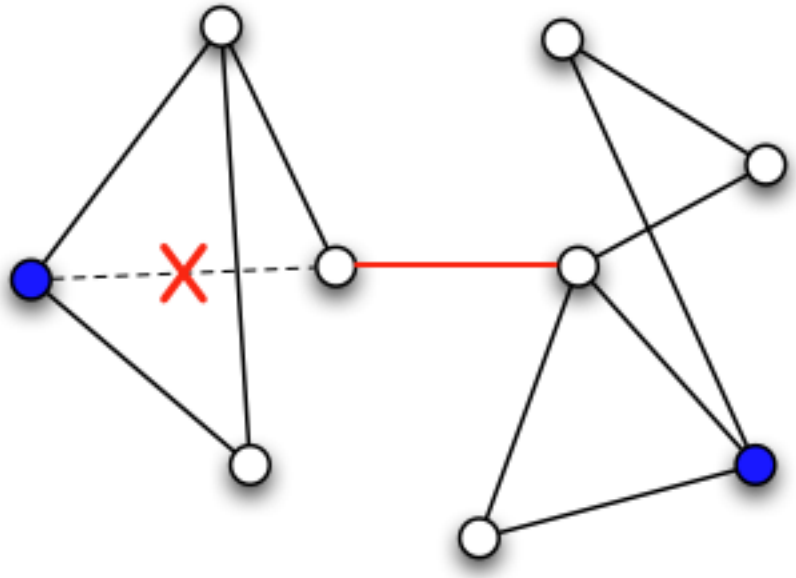
- We can insert or delete edges in this graph, and still find the connectivity of any pair of vertices





Dynamic Connectivity

- We can insert or delete edges in this graph, and still find the connectivity of any pair of vertices

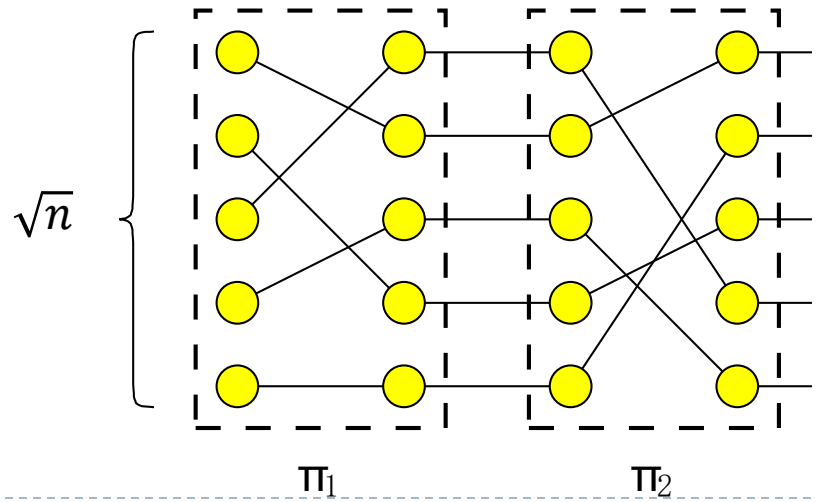




Reduction from partial sum to dynamic connectivity

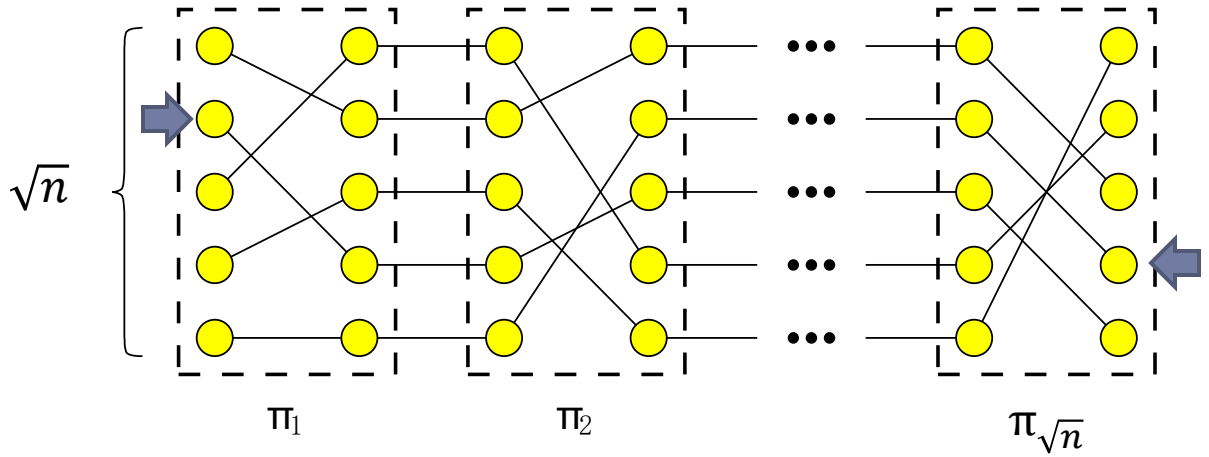
- Our lower bound for partial sum holds for any group $\mathbb{G}$
- We choose $\mathbb{G}=S_{\sqrt{n}}$, the symmetric group on $\sqrt{n}$ letters
  - every $\pi \in \mathbb{G}$ is a permutation on $(1,2,3,\ldots,\sqrt{n})$

- The sum is defined as their composition $\pi_i \circ \pi_j$,
  - looks like:



Reduction from partial sum to dynamic connectivity

- Given $\sqrt{n}$ permutations in $\mathbb{G}$
  - Then the sum $\boldsymbol{\pi}_1 \circ \ldots \circ \boldsymbol{\pi}_i$ is a permutation as well
  - We can construct a $\sqrt{n} \times \sqrt{n}$ grid:

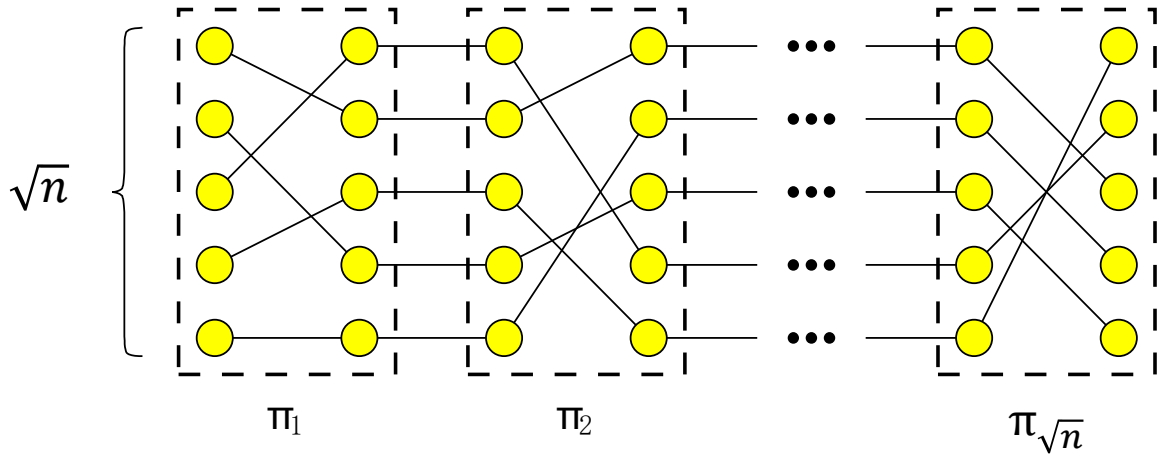


  - Given $\boldsymbol{\sigma} \in \mathbb{G}$, checking whether $\boldsymbol{\sigma} = \boldsymbol{\pi}_1 \circ \ldots \circ \boldsymbol{\pi}_i$ can be obtained by checking $\sqrt{n}$ connectivity queries



Reduction from partial sum to dynamic connectivity

- Given $\sqrt{n}$ permutations in $\mathbb{G}$
  - Then the sum $\pi_1 \circ \ldots \circ \pi_i$ is a permutation as well
  - We can construct a $\sqrt{n} \times \sqrt{n}$ grid:



- $\delta = \log|S_{\sqrt{n}}| \sim \sqrt{n}\log n$, so our lower bound for partial sum takes $\Omega\left(\frac{\delta}{w}\log n\right) = \Omega(\sqrt{n}\log n)$. (we take $w = \boldsymbol{\Theta}(\log n)$)



- $\log|S_{\sqrt{n}}| \approx \sqrt{n}\log n$, so our lower bound for partial sum takes $\Omega(\sqrt{n}\log n)$
  - However, there is no obvious way to perform sum by connected queries, so we need to change SUM operation to *(with the same lower bound):*

    Verify-Sum(i, σ): whether $\sigma = \pi_1 \circ \ldots \circ \pi_i$

Thus, if we can check $\sqrt{n}$ connectivity queries in each o(log n) time, we can do Verify-Sum in $o(\sqrt{n}\log n)$ time

- **Ω(log n) lower bound on dynamic connectivity**
- For details, see *"LOGARITHMIC LOWER BOUNDS IN THE CELL-PROBE MODEL"* by Pătraşcu and Demaine



Summary

- Were-you-last game
- Yao's cell probe model
- Ramsey theorem
- Lower bounds on dynamic partial sum

